



Syrian Private University
Faculty of Artificial Intelligence
Engineering
Department of Intelligent Information
Security Systems



Enhancing IDS by Integrating SVM and RF Using Dempster-Shafer Theory

Project Report (Junior Project) submitted in partial fulfillment of the requirements for the Bachelor Degree in Artificial Intelligence Engineering - Intelligent Information Security Systems

Prepared by:

Hasan Obaid
Ahmed Alhussien

Supervised by:

Dr. Wassim Juneidi
Eng. Ahmed Sheikha

Academic Year: 2025-2026

تعزيز أنظمة كشف التسلل من خلال دمج خوارزميتي SVM و RF باستخدام Dempster-Shafer Theory

تقرير مشروع (فصلي) – قُدِّم استكمالاً لمتطلبات الحصول على درجة الإجازة في

هندسة الذكاء الاصطناعي- هندسة نظم أمن المعلومات الذكية

إعداد:

حسن العبيد

أحمد الحسين

إشراف:

د . وسيم جنيدي

م .أحمد شيخة

العام الدراسي: 2025-2026

Dedication

This work is dedicated to our families, instructors, and everyone who supported our academic journey. Their encouragement and guidance have been a continuous source of motivation.

Table of Contents

List of Tables	7
List of Figures.....	8
List of Abbreviations and Scientific Terms	9
Abstract.....	10
Arabic Abstract.....	11
Introduction.....	12
Chapter One: Theoretical Framework	14
1-1. Introduction	15
1-2. Intrusion Detection Systems	15
1-2-1. Definition and Purpose of IDS.....	15
1-2-2. IDS Categories	15
1-3. Traditional IDS and Machine Learning-Based IDS.....	15
1-4. Machine Learning Classification in IDS	16
1-5. Support Vector Machine	16
1-5-1. Principle of SVM	16
1-5-2. SVM in Intrusion Detection	16
1-6. Random Forest	17
1-6-1. Principle of Random Forest	17
1-6-2. RF in IDS Applications.....	17
1-7. Ensemble Learning and Classifier Fusion	18
1-8. Dempster-Shafer Theory	19
1-8-1. Concept of Dempster-Shafer Theory	19
1-8-2. Frame of Discernment.....	19
1-8-3. Basic Belief Assignment.....	19
1-8-4. Conflict Coefficient and Combination Rule	20
1-9. Data Preprocessing in IDS	20
1-10. Feature Selection Using LASSO.....	21
1-11. Class Imbalance and SMOTE	22
1-12. CSE-CIC-IDS2018 Dataset.....	23
1-13. Evaluation Metrics	23
1-14. Chapter Summary	25
Chapter Two: Related Work	26
2-1. Introduction	27
2-2. Development of Machine Learning-Based IDS.....	27
2-3. Ensemble Learning Approaches in IDS.....	27
2-4. Evidence Fusion and Dempster-Shafer-Based Approaches.....	28
2-5. Comparative Analysis of Related Studies.....	29
2-5-1. Summary Table of Related Studies.....	29
2-6. Research Gap	30
2-7. Position of the Current Project.....	31
2-8. Chapter Summary	31
Chapter Three: Proposed Methodology	32
3-1. Introduction	33

3-2. General Architecture of the Proposed IDS	33
3-2-1. Main Stages of the Architecture.....	33
3-3. Dataset Preparation	34
3-4. Data Preprocessing.....	34
3-5. Feature Selection Using LASSO.....	35
3-6. Class Balancing Using SMOTE.....	35
3-7. Support Vector Machine Classification	36
3-8. Random Forest Classification	36
3-9. Dempster-Shafer-Based Decision Fusion	37
3-9-1. Frame of Discernment.....	37
3-9-2. Conversion to Belief Assignments.....	37
3-9-3. Conflict Coefficient.....	37
3-9-4. Final Decision Rule.....	37
3-10. Proposed Algorithm	38
3-10-1. Algorithm Specification	38
3-10-2. Algorithm Flowchart.....	38
3-11. Experimental and Implementation Configuration.....	39
3-12. Chapter Summary	40
Chapter Four: Practical Implementation	42
4-1. Introduction	43
4-2. Practical Work Environment.....	43
4-3. Dataset Used in the Practical Implementation	43
4-4. Dataset Loading and Initial Preparation.....	44
4-4-1. Library Import and Dataset Loading.....	44
4-4-2. Label Cleaning, Feature Preparation, and Train-Test Split	44
4-5. Data Balancing, Feature Selection, and Scaling	45
4-5-1. Class Balancing Using SMOTE.....	45
4-5-2. LASSO Feature Selection	46
4-5-3. Feature Scaling.....	46
4-6. Model Training Implementation	46
4-6-1. Support Vector Machine Training	46
4-6-2. Random Forest Training and Probability Outputs	47
4-7. Evaluation and Dempster-Shafer Fusion Implementation	47
4-7-1. Evaluation Function	47
4-7-2. Conversion to Belief Assignments.....	48
4-7-3. Dempster-Shafer Combination and Final Decision	48
4-8. Final Evaluation and Debugging Outputs.....	49
4-9. Chapter Summary	50
Chapter Five: Results and Discussion.....	51
5-1. Introduction	52
5-2. Dataset and Processing Output	52
5-2-1. Dataset Split and Class Distribution	52
5-2-2. Feature Selection Output.....	52
5-3. Performance Results of the Implemented Models	52
5-3-1. SVM Individual Results.....	53
5-3-2. Random Forest Individual Results.....	53

5-3-3. Hybrid SVM-RF-DST Results.....	53
5-4. Confusion Matrix Analysis	54
5-5. Error Analysis and Operational Interpretation.....	55
5-6. Comparison with the Primary Study	56
5-7. Discussion	57
5-8. Chapter Summary	57
Conclusion and Future Work	58
Project Conclusion	58
Main Contributions.....	58
Limitations.....	58
Future Work	58
References	60

List of Tables

Table 1: Summary and Critical Analysis of Related Studies.....	29
Table 2: Experimental and Implementation Configuration of the Proposed IDS [1]	39
Table 3: Algorithmic Components and Their Roles in the Proposed IDS.....	40
Table 4: Inputs and Outputs of the Main Processing Stages	41
Table 5: Practical Work Environment and Dataset.....	43
Table 6: Performance Results of the Implemented Models.....	52
Table 7: Confusion Matrix Values of the Implemented Models	54
Table 8: Comparison Between Implemented Results and the Primary Study	56

List of Figures

Figure 1: Support Vector Machine Classification Concept	17
Figure 2: Random Forest Classification Structure	18
Figure 3: Conceptual Workflow of the Proposed Hybrid IDS	19
Figure 4: Dempster-Shafer Evidence Fusion Process.....	20
Figure 5: Data Preprocessing Pipeline	21
Figure 6: LASSO Feature Selection Process	22
Figure 7: SMOTE Class Balancing Process	23
Figure 8: Confusion Matrix Structure	25
Figure 9: General Architecture of the Proposed Hybrid IDS	34
Figure 10: Complete Proposed Algorithm Flowchart.....	39
Figure 11: Library Import and Dataset Loading Code.....	44
Figure 12: Label Cleaning, Feature Preparation, and Train-Test Split Code	45
Figure 13: SMOTE Class Balancing Code	45
Figure 14: LASSO Feature Selection Code	46
Figure 15: Feature Scaling Code.....	46
Figure 16: SVM Training and Probability Calibration Code.....	47
Figure 17: Random Forest Training and Probability Output Code.....	47
Figure 18: Model Evaluation Function Code.....	48
Figure 19: Belief Assignment Construction Code	48
Figure 20: Dempster-Shafer Combination and Final Decision Code	49
Figure 21: Final Evaluation, Summary Table, and Fusion Debugging Code	49
Figure 22: Performance Comparison of Implemented Models.....	53
Figure 23: Confusion Matrix of the SVM Model	54
Figure 24: Confusion Matrix of the Random Forest Model	54
Figure 25: Confusion Matrix of the Hybrid SVM-RF-DST Model.....	55
Figure 26: False Positive and False Negative Comparison	55
Figure 27: Implemented Hybrid Model Compared with the Primary Study	56

List of Abbreviations and Scientific Terms

The following table lists the abbreviations and scientific terms used in this report. The abbreviations are arranged alphabetically.

Abbreviation	Scientific Term
ANN	Artificial Neural Network
BBA	Basic Belief Assignment
CSE-CIC-IDS2018	Communications Security Establishment and Canadian Institute for Cybersecurity Intrusion Detection System 2018
DST	Dempster-Shafer Theory
FN	False Negative
FP	False Positive
IDS	Intrusion Detection System
LASSO	Least Absolute Shrinkage and Selection Operator
ML	Machine Learning
RF	Random Forest
SMOTE	Synthetic Minority Oversampling Technique
SVM	Support Vector Machine
TN	True Negative
TP	True Positive

Abstract

Intrusion Detection Systems play a critical role in protecting computer networks from unauthorized access, malicious activities, and evolving cyberattacks. Traditional intrusion detection approaches, especially signature-based and rule-based systems, are effective against known attacks but often fail to detect new or modified threats. They may also suffer from high false alarm rates when network traffic patterns are complex or ambiguous. For this reason, machine learning techniques have become an important direction for improving the accuracy and adaptability of intrusion detection systems.

This project focuses on a hybrid intrusion detection approach that integrates Support Vector Machine and Random Forest classifiers using Dempster-Shafer Theory. The main objective is to improve detection reliability by combining the strengths of two complementary machine learning models. Support Vector Machine contributes strong decision-boundary construction, while Random Forest provides ensemble-based robustness and the ability to handle complex feature relationships. Dempster-Shafer Theory is used as an evidence-fusion method to combine the outputs of both classifiers and reduce uncertainty in the final decision.

The adopted methodology is based on the CSE-CIC-IDS2018 dataset, which contains labeled benign and malicious network traffic flows. The data processing pipeline includes data cleaning, feature standardization, feature selection using Least Absolute Shrinkage and Selection Operator, and class balancing using the Synthetic Minority Oversampling Technique. After training the Support Vector Machine and Random Forest models, their probabilistic outputs are transformed into belief assignments and fused using Dempster-Shafer Theory to produce the final classification decision.

Keywords: Intrusion Detection System, Support Vector Machine, Random Forest, Dempster-Shafer Theory, Machine Learning, CSE-CIC-IDS2018, Feature Selection, SMOTE.

Arabic Abstract

تؤدي أنظمة كشف التسلل دوراً مهماً في حماية الشبكات الحاسوبية من الوصول غير المصرح به، والأنشطة الخبيثة، والهجمات السيبرانية المتطورة. تُعد الأساليب التقليدية لكشف التسلل، ولا سيما الأنظمة المعتمدة على التوافيق والقواعد، فعالة في اكتشاف الهجمات المعروفة، إلا أنها غالباً ما تكون محدودة في اكتشاف الهجمات الجديدة أو المعدلة. كما قد تعاني هذه الأنظمة من ارتفاع معدلات الإنذارات الخاطئة عندما تكون أنماط حركة المرور الشبكية معقدة أو غير واضحة. لذلك أصبحت تقنيات التعلم الآلي اتجاهاً مهماً لتحسين دقة أنظمة كشف التسلل وقدرتها على التكيف.

يركز هذا المشروع على منهجية هجينة لكشف التسلل تعتمد على دمج مصنفي **SVM** و **RF** باستخدام **DST**. يهدف المشروع إلى تحسين موثوقية الكشف من خلال الجمع بين مزايا نموذجين متكاملين من نماذج التعلم الآلي. يساهم **SVM** في بناء حدود فصل فعالة بين الفئات، في حين يوفر **RF** متانة أعلى بفضل اعتماده على التعلم التجميعي وقدرته على التعامل مع العلاقات المعقدة بين الميزات. وتُستخدم **DST** كآلية لدمج الأدلة من أجل الجمع بين مخرجات المصنفين وتقليل عدم اليقين في القرار النهائي.

تعتمد المنهجية المتبعة على مجموعة البيانات **CSE-CIC-IDS2018**، التي تحتوي على تدفقات حركة مرور شبكية موسومة كحركة مرور سليمة أو خبيثة. تشمل مراحل معالجة البيانات تنظيف البيانات، وتقييس الميزات، واختيار الميزات باستخدام خوارزمية **LASSO**، ومعالجة عدم توازن الفئات باستخدام تقنية **SMOTE**. بعد تدريب مصنفي **SVM** و **RF**، تُحوّل مخرجاتهما الاحتمالية إلى قيم اعتقاد، ثم تُدمج باستخدام **DST** لإنتاج قرار التصنيف النهائي.

الكلمات المفتاحية: نظام كشف التسلل، **DST**، **RF**، **SVM**، التعلم الآلي، **CSE-CIC-IDS2018**، اختيار الميزات، **SMOTE**

Introduction

Cybersecurity has become one of the most essential requirements for protecting modern digital environments. The continuous expansion of computer networks, cloud services, online applications, and connected systems has increased the number of possible attack surfaces. As a result, cyberattacks have become more frequent, more complex, and more difficult to detect using traditional security mechanisms. Network intrusions may lead to data theft, service disruption, unauthorized access, financial losses, and damage to institutional reputation. Therefore, detecting abnormal and malicious activities within network traffic is a critical task in modern cybersecurity.

Intrusion Detection Systems (IDSs) are designed to monitor network activities and identify suspicious behavior that may indicate attacks or policy violations. Traditional IDS approaches are often based on predefined signatures or manually written rules. Although these methods are useful for detecting known attacks, they are less effective against new, modified, or zero-day threats. They may also produce high false alarm rates when network behavior changes or when attack patterns do not exactly match stored signatures. This limitation has encouraged researchers to adopt machine learning techniques that can learn patterns from data and identify complex relationships in network traffic [1], [4].

Machine learning-based IDS models provide an adaptive alternative to traditional detection systems. These models can be trained on labeled network traffic data to distinguish between normal and malicious behavior. However, relying on a single classifier is often insufficient in real-world network environments. Single models may be affected by class imbalance, overlapping traffic patterns, uncertainty in network behavior, and variability in attack types. For example, a classifier may perform well on one attack category while producing weaker results on another. This issue becomes more challenging when large-scale datasets contain millions of records and multiple attack classes [1].

To address these limitations, the current project focuses on a hybrid intrusion detection approach based on integrating two machine learning classifiers: Support Vector Machine (SVM) and Random Forest (RF). SVM is known for its ability to construct effective decision boundaries, especially in high-dimensional spaces, while RF is an ensemble learning model that improves robustness by combining multiple decision trees. The combination of these two models can provide complementary strengths: SVM contributes margin-based classification, whereas RF contributes feature diversity and ensemble-based generalization [1], [4].

The main distinguishing element of the proposed approach is the use of Dempster-Shafer Theory (DST) as an evidence fusion method. Instead of applying simple voting or averaging between classifier outputs, DST provides a formal framework for combining uncertain and possibly conflicting evidence. In this context, the outputs of SVM and RF are transformed into belief assignments, then fused to produce a final intrusion detection decision. This makes the decision-making process more suitable for uncertain cybersecurity environments, where classifier outputs may not always fully agree [1], [2], [3].

The project is based on the CSE-CIC-IDS2018 dataset, which contains labeled benign and attack network traffic flows. The main study used preprocessing techniques such as data

cleaning, standardization, class balancing using SMOTE, and feature selection using LASSO. The selected features were then used to train and evaluate SVM and RF classifiers, followed by DST-based decision fusion. The reported results showed that the DST fusion model outperformed the individual classifiers.

The main objective of this project is to study, implement, and present a hybrid IDS framework that improves detection reliability by combining SVM and RF through Dempster-Shafer Theory. The project also aims to explain the theoretical basis of the used algorithms, analyze related research, and clarify why evidence-based fusion can be more suitable than relying on a single classifier or simple ensemble voting. The project contributes to the field of intelligent cybersecurity systems by presenting an approach that handles uncertainty, improves classification robustness, and supports more reliable intrusion detection decisions.

This report is organized as follows. Chapter One presents the theoretical framework required to understand the project, including IDS concepts, machine learning-based intrusion detection, SVM, Random Forest, ensemble learning, Dempster-Shafer Theory, data preprocessing, feature selection, class balancing, the CSE-CIC-IDS2018 dataset, and evaluation metrics. Chapter Two reviews previous studies related to ensemble learning, hybrid IDS models, and evidence fusion approaches. Chapter Three presents the proposed methodology and the implementation configuration. Chapter Four explains the practical implementation environment and code workflow. Chapter Five presents the experimental results and compares them with the primary study. The report ends with the conclusion and future work.

Chapter One: Theoretical Framework

1-1. Introduction

This chapter presents the theoretical concepts required to understand the hybrid intrusion detection approach used in this project. The chapter begins by explaining Intrusion Detection Systems and the transition from traditional IDS methods to machine learning-based detection. It then discusses the two main classifiers used in the project, namely Support Vector Machine and Random Forest. After that, the chapter explains ensemble learning and Dempster-Shafer Theory as a formal method for evidence fusion. The chapter also presents the preprocessing techniques, feature selection method, class balancing approach, dataset, and evaluation metrics used to assess IDS performance.

1-2. Intrusion Detection Systems

1-2-1. Definition and Purpose of IDS

An Intrusion Detection System (IDS) is a security mechanism used to monitor computer systems or network traffic in order to identify suspicious, abnormal, or malicious activities. IDSs are considered important security components because they support early detection of attacks and help administrators respond before serious damage occurs. In network environments, IDSs analyze traffic behavior and attempt to distinguish normal activity from possible intrusions or policy violations [1], [4].

1-2-2. IDS Categories

IDSs can be generally categorized into host-based and network-based systems. A host-based IDS monitors activities on a specific device, such as system logs, file changes, and process behavior. A network-based IDS analyzes traffic flowing through a network and attempts to detect signs of malicious activity in packets or flows. In the context of this project, the focus is on network intrusion detection, where traffic flow features are used to classify activities as benign or malicious [1], [4].

Traditional IDSs often depend on signatures and rules. Signature-based detection compares observed traffic with known attack patterns. This approach is efficient for detecting previously identified threats, but it cannot effectively detect new attacks whose signatures are not stored in the system. Rule-based systems face a similar limitation because they depend on manually designed rules that may not cover all possible attack variations. These limitations have encouraged the use of machine learning methods to create IDS models that can learn patterns automatically from data [1], [4].

1-3. Traditional IDS and Machine Learning-Based IDS

Traditional IDS approaches are useful but limited in dynamic cybersecurity environments. Signature-based systems are highly dependent on existing databases of known attack patterns. When attackers modify their behavior or use new techniques, traditional IDSs may fail to detect the attack. In addition, such systems may generate false positives when legitimate behavior resembles suspicious activity.

Machine learning-based IDSs attempt to overcome these limitations by learning from network traffic data. Instead of relying only on fixed signatures, machine learning models analyze statistical and behavioral features of traffic. These features may include packet lengths,

flow duration, header information, protocol flags, inter-arrival times, and throughput-related measurements. Once trained, the model can classify new traffic as normal or malicious.

Machine learning improves IDS adaptability because it enables the system to recognize complex patterns that may not be easily expressed as manual rules. However, machine learning also introduces challenges. The performance of a model depends on the quality of training data, preprocessing, feature selection, class balance, and the suitability of the chosen algorithm. A single model may perform well under some conditions but poorly under others. Therefore, hybrid and ensemble methods are often used to improve robustness and reduce the limitations of individual classifiers [1], [4].

1-4. Machine Learning Classification in IDS

In intrusion detection, classification is the process of assigning network traffic records to predefined classes. In binary classification, the system classifies each record as either benign or malicious. In multi-class classification, the system assigns records to more specific categories, such as DoS, DDoS, brute-force, botnet, infiltration, or SQL injection.

The current project follows the binary classification setting used in the main study, where benign traffic is assigned to one class and all attack categories are grouped into another class. This setting reflects a practical IDS scenario in which the main objective is to determine whether a network flow is safe or suspicious [1].

Machine learning classification depends on extracting meaningful features from traffic records. These features are then used to train a model that learns the relationship between input values and class labels. During testing, the model receives unseen traffic records and predicts the corresponding class. The quality of classification is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrix analysis [1], [4].

1-5. Support Vector Machine

1-5-1. Principle of SVM

Support Vector Machine is a supervised learning algorithm used for classification and regression tasks. In classification problems, SVM attempts to find the best decision boundary that separates data points belonging to different classes. This decision boundary is called a hyperplane. The optimal hyperplane is selected in a way that maximizes the margin between the closest data points of different classes [1].

1-5-2. SVM in Intrusion Detection

In intrusion detection, SVM is useful because network traffic data may contain high-dimensional features. SVM can handle such feature spaces effectively, especially when the classes can be separated by a clear boundary. When data is not linearly separable, kernel functions can be used to map the data into a higher-dimensional space where separation becomes easier.

The main advantage of SVM is its ability to construct strong decision boundaries. This makes it suitable for distinguishing between benign and malicious traffic patterns. However, SVM can be computationally expensive when trained on very large datasets. For this reason, the main study used GPU acceleration through RAPIDS cuML to train and optimize the SVM model on the large CSE-CIC-IDS2018 dataset [1].

SVM performance is also sensitive to feature scaling. If features have different numerical ranges, the decision boundary may be influenced by features with larger values. Therefore, standardization is an important preprocessing step before training SVM. In the main approach, standardization was applied to ensure that input features had consistent numerical distributions [1].

Figure 1 illustrates the SVM classification concept by showing how an optimal hyperplane separates benign and attack samples with a maximum margin.

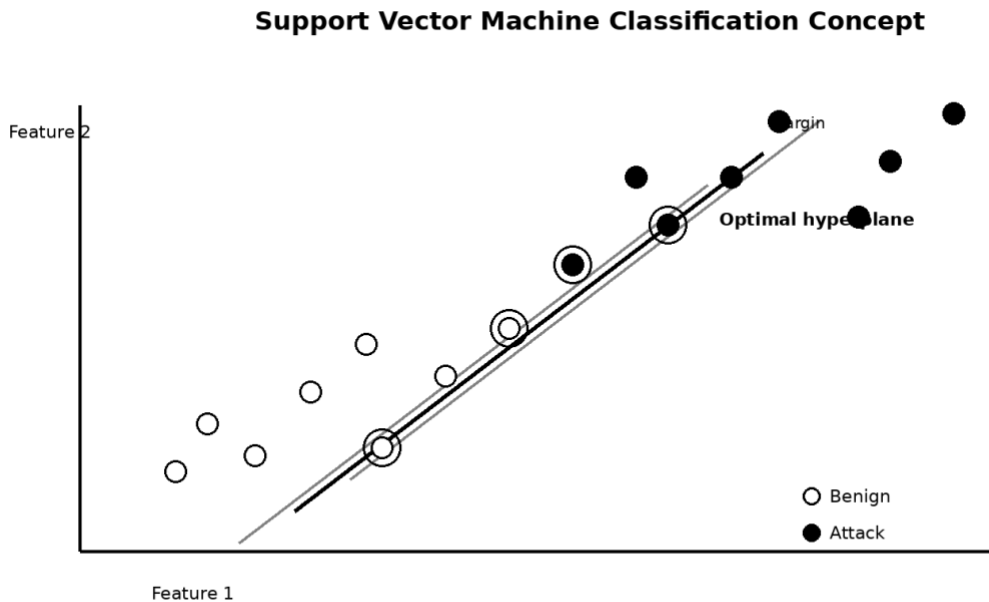


Figure 1: Support Vector Machine Classification Concept

1-6. Random Forest

1-6-1. Principle of Random Forest

Random Forest is an ensemble learning algorithm based on constructing multiple decision trees. Each tree is trained on a different subset of the data, and the final decision is obtained by aggregating the predictions of all trees. In classification tasks, the class predicted by the majority of trees is usually selected.

The strength of Random Forest comes from diversity. Since each tree is trained using different samples and feature subsets, the model reduces the risk of overfitting compared with a single decision tree. It can handle nonlinear relationships and complex interactions between features, which makes it suitable for intrusion detection datasets [1], [4].

1-6-2. RF in IDS Applications

In IDS applications, RF is useful because network traffic can contain many different types of features. Some attacks may be reflected in packet length statistics, while others may appear in flag counts, flow duration, or inter-arrival time patterns. Random Forest can capture these different feature relationships through its ensemble of trees [1].

Compared with SVM, Random Forest is generally more flexible in handling complex datasets and may be less sensitive to feature scaling. However, its performance can still be affected by class imbalance and noisy features. Therefore, preprocessing, feature selection, and balancing techniques remain important for improving RF performance in IDS tasks [1], [4].

Figure 2 shows how Random Forest combines several decision trees and aggregates their outputs to produce a final classification decision.

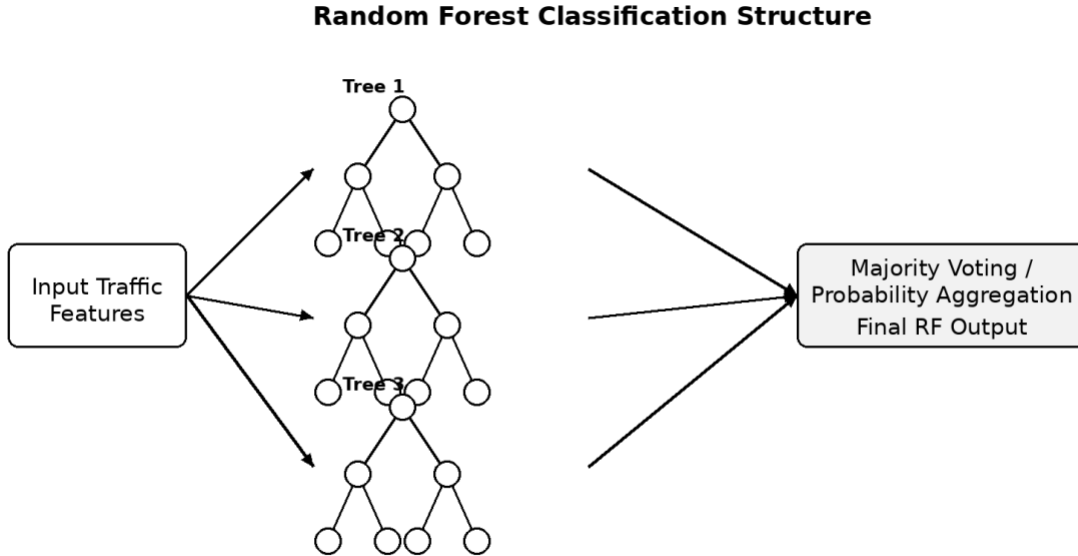


Figure 2: Random Forest Classification Structure

1-7. Ensemble Learning and Classifier Fusion

Ensemble learning refers to combining multiple models to improve prediction performance. The main idea is that different models may make different errors, and combining them can produce a more reliable final decision. Ensemble methods can reduce variance, improve generalization, and increase robustness in complex classification tasks [1], [4].

There are several ensemble strategies. Voting combines predictions from multiple classifiers by selecting the class with the majority vote. Averaging combines probability outputs by computing their average. Stacking uses a meta-classifier to learn how to combine the outputs of base classifiers. Although these methods are useful, they may not explicitly model uncertainty or conflict between classifiers [1], [2], [3].

In intrusion detection, uncertainty is an important issue because network traffic patterns can be ambiguous. For example, one classifier may assign a flow to the attack class, while another classifier may assign it to the benign class. Simple voting may ignore the degree of confidence behind each decision. Therefore, evidence fusion methods such as Dempster-Shafer Theory provide a more formal mechanism for combining classifier outputs under uncertainty [1], [2], [3].

Figure 3 summarizes the conceptual workflow that connects preprocessing, feature selection, classifier outputs, DST fusion, and the final IDS decision.

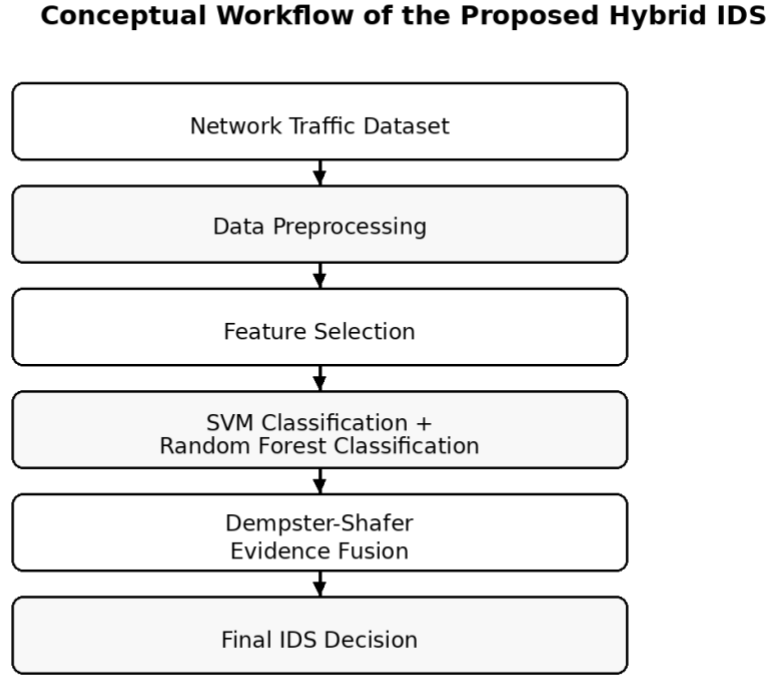


Figure 3: Conceptual Workflow of the Proposed Hybrid IDS

1-8. Dempster-Shafer Theory

1-8-1. Concept of Dempster-Shafer Theory

Dempster-Shafer Theory is a mathematical framework for reasoning under uncertainty. It is also known as evidence theory or the theory of belief functions. Unlike classical probability theory, which assigns probability values to individual events, DST allows belief to be assigned to sets of hypotheses. This makes it useful when evidence is incomplete, uncertain, or conflicting [2],[3].

1-8-2. Frame of Discernment

In the context of this project, the frame of discernment can be defined as follows:

$$\Theta = \{\text{Benign}, \text{Attack}\} \quad (1)$$

This means that the IDS decision is made between two possible hypotheses: normal traffic and malicious traffic. Each classifier provides evidence supporting one of these hypotheses. The output probabilities of SVM and RF can be transformed into Basic Belief Assignments. These belief values represent how strongly each classifier supports the benign or attack class.

1-8-3. Basic Belief Assignment

A simplified belief assignment can be written as follows:

$$m(\{0\}) = 1 - p_{\text{attack}}, \quad m(\{1\}) = p_{\text{attack}}, \quad m(\Theta) = 0 \quad (2)$$

where $\{0\}$ represents the benign class, $\{1\}$ represents the attack class, and Θ represents uncertainty. The value p_{attack} represents the probability assigned by a classifier to the attack class.

1-8-4. Conflict Coefficient and Combination Rule

When two classifiers provide evidence, DST combines their belief assignments using Dempster's rule of combination. The conflict coefficient K measures the disagreement between the two evidence sources:

$$K = m_A(\{0\})m_B(\{1\}) + m_A(\{1\})m_B(\{0\}) \quad (3)$$

where m_A and m_B represent the belief assignments of two classifiers. A high value of K indicates stronger conflict between the classifiers. The combined belief for the benign and attack classes is then computed by redistributing non-conflicting evidence. This process allows the final decision to reflect both agreement and disagreement between classifiers.

DST is particularly suitable for IDS because classifier outputs may be uncertain or contradictory. In the main study, the outputs of SVM and RF were converted into belief assignments and then fused using DST. The final class was selected based on the highest belief value after fusion [1]. This method provides a more structured fusion mechanism than simple voting or averaging.

Figure 4 clarifies how SVM and RF probabilities are converted into belief assignments and fused using Dempster-Shafer Theory.

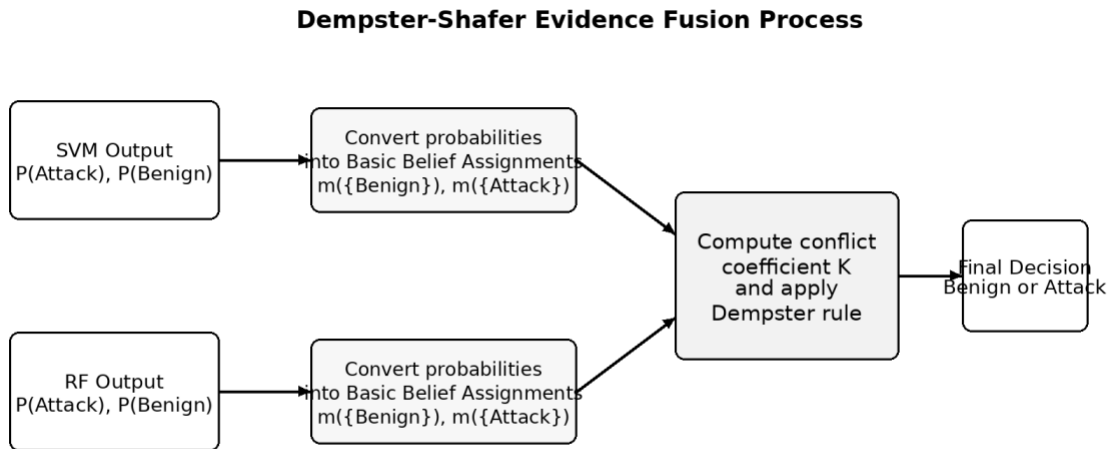


Figure 4: Dempster-Shafer Evidence Fusion Process

1-9. Data Preprocessing in IDS

Data preprocessing is a necessary stage before training machine learning models. Raw intrusion detection datasets may contain missing values, duplicated records, non-numeric values, irrelevant identifiers, and inconsistent feature ranges. If these issues are not handled properly, model performance may be negatively affected.

The preprocessing stage in the main study included cleaning the dataset, handling missing and infinite values, removing non-informative identifiers, encoding labels, and standardizing numerical features. Standardization is especially important for SVM because the model depends on distances and margins between data points. If features are not scaled, values with larger numerical ranges may dominate the classification process [1].

Preprocessing also improves the reliability of Random Forest and DST fusion. Clean and consistent input data allows classifiers to generate more meaningful probability outputs, which are later transformed into belief assignments for DST. Therefore, preprocessing is not only a preparation step but also a key factor in improving the quality of the final IDS decision.

Figure 5 presents the preprocessing pipeline that prepares raw network traffic records before feature selection and model training.

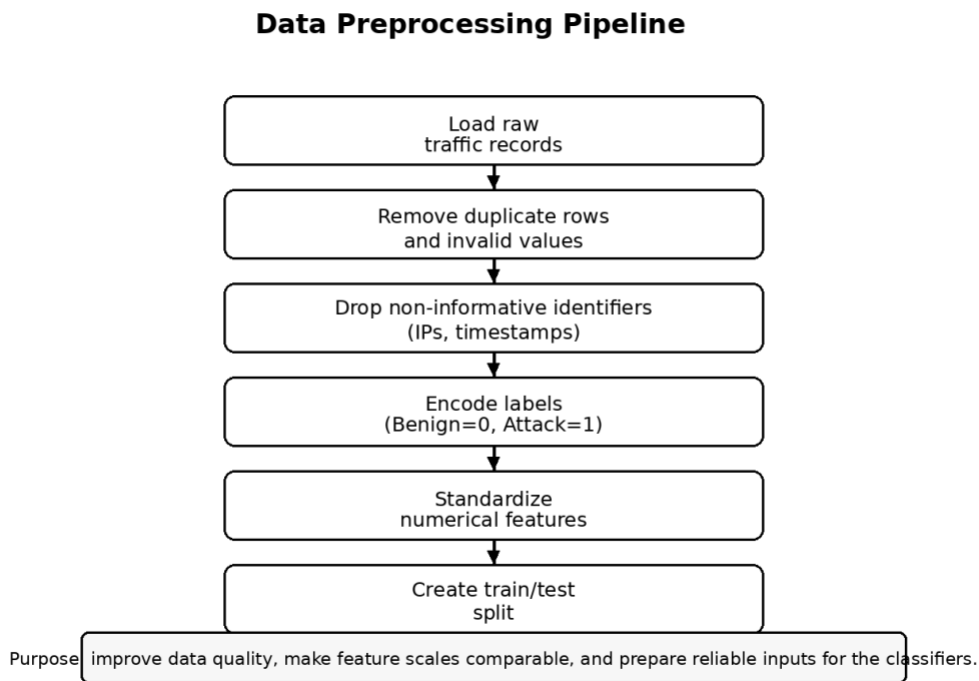


Figure 5: Data Preprocessing Pipeline

1-10. Feature Selection Using LASSO

Feature selection is the process of selecting the most important features from the dataset while removing redundant or weak predictors. This step is important because intrusion detection datasets may contain a large number of features, not all of which contribute equally to classification.

The main study used the Least Absolute Shrinkage and Selection Operator, known as LASSO, for feature selection. LASSO is based on L1 regularization, which can reduce the coefficients of less important features to zero. As a result, only the most relevant features are retained.

In the main study, the original 78 numerical features of the CSE-CIC-IDS2018 dataset were reduced to 46 selected features. These selected features included packet length statistics, flow

inter-arrival time, header length, flag counts, active and idle flow information, and throughput-related measurements [1]. Reducing the number of features improves computational efficiency and makes the model more interpretable. It also reduces noise and redundancy, which can improve classification performance.

Feature selection is particularly important in large-scale IDS datasets because training models on millions of records with many features can be computationally expensive. By selecting only the most informative features, the system becomes more efficient while preserving the discriminative information needed to detect attacks.

Figure 6 illustrates the role of LASSO in reducing the original feature space to the most discriminative traffic features.

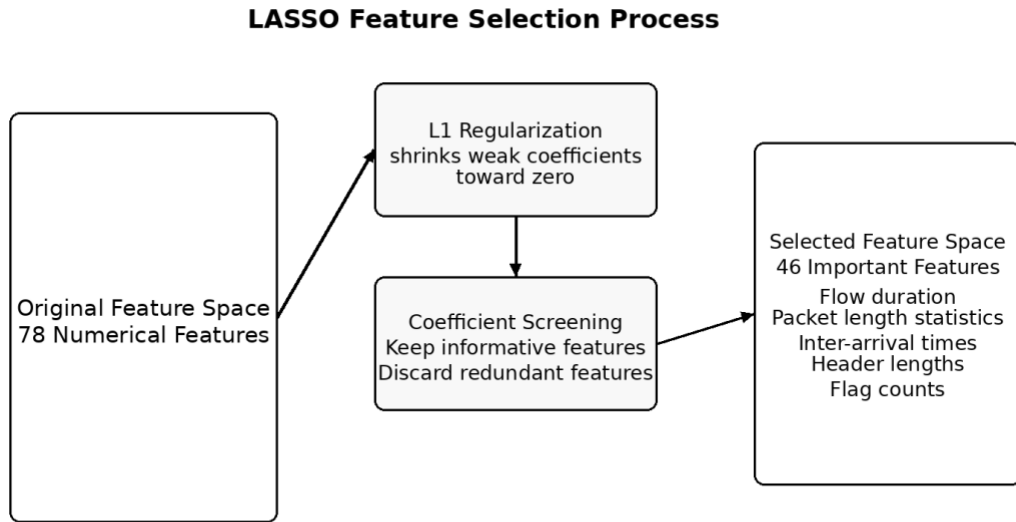


Figure 6: LASSO Feature Selection Process

1-11. Class Imbalance and SMOTE

Class imbalance is a common problem in intrusion detection datasets. In real network environments, benign traffic is usually much more frequent than attack traffic. If a classifier is trained on imbalanced data, it may become biased toward the majority class. As a result, the model may achieve high overall accuracy while failing to detect important attack samples.

In the main study, the dataset distribution showed a higher percentage of benign samples compared with attack samples. To address this problem, the Synthetic Minority Oversampling Technique was applied to the training data. SMOTE generates synthetic samples for the minority class rather than simply duplicating existing samples. This helps the model learn a better decision boundary for the minority class.

Balancing the dataset improves the ability of classifiers to detect attacks. It also supports more reliable DST fusion because both SVM and RF can produce better probability estimates

when trained on balanced data. In the main approach, SMOTE was applied before classifier training to reduce the negative impact of class imbalance [1].

Figure 7 explains the role of SMOTE in reducing class imbalance by generating synthetic minority-class attack samples in the training data.

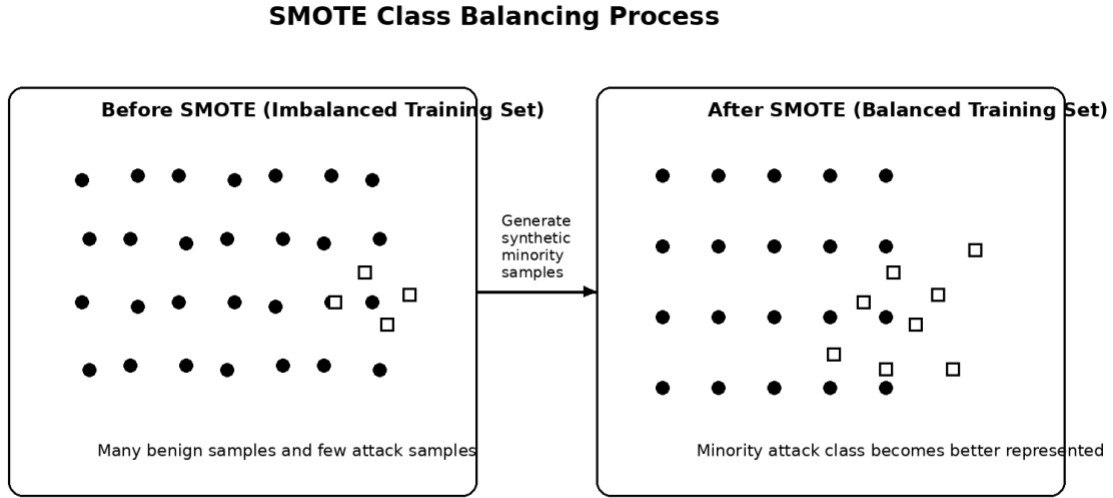


Figure 7: SMOTE Class Balancing Process

1-12. CSE-CIC-IDS2018 Dataset

The CSE-CIC-IDS2018 dataset is a large-scale intrusion detection dataset developed to represent realistic network traffic conditions. It contains labeled benign traffic and multiple attack categories, making it suitable for evaluating machine learning-based IDS models. The dataset includes different attack scenarios such as DDoS-HOIC, LOIC-UDP, SSH brute-force, SQL injection, and infiltration [1].

The main study used a merged version of the dataset containing approximately 7.2 million network flow records and 78 numerical features. These features describe different characteristics of network flows, such as packet lengths, flow duration, header information, inter-arrival times, and protocol-related indicators [1].

This dataset is suitable for the current project because it provides a realistic and diverse evaluation environment. It includes both normal and malicious traffic and reflects the complexity of modern intrusion detection tasks. The use of a large dataset also allows the proposed hybrid model to be tested under conditions that are closer to real-world cybersecurity environments.

1-13. Evaluation Metrics

Evaluation metrics are used to measure the performance of intrusion detection models. In binary IDS classification, the confusion matrix consists of four values: True Positive, True Negative, False Positive, and False Negative. TP represents attacks correctly classified as attacks. TN represents benign traffic correctly classified as benign. FP represents benign traffic incorrectly classified as attack. FN represents attack traffic incorrectly classified as benign.

Accuracy measures the overall proportion of correct predictions:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \quad (4)$$

Precision measures how reliable the positive attack predictions are:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \quad (5)$$

Recall measures how effectively the model detects actual attacks:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \quad (6)$$

F1-score is the harmonic mean of precision and recall:

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (7)$$

In IDS evaluation, accuracy alone may be misleading when the dataset is imbalanced. A model may classify most samples as benign and still obtain high accuracy, while missing many attacks. Therefore, precision, recall, and F1-score are also necessary. Precision is important because false alarms waste analysts' time and may disrupt legitimate traffic. Recall is important because missed attacks can cause serious security incidents. F1-score provides a balanced measure between precision and recall.

The main study used accuracy, precision, recall, F1-score, and confusion matrix analysis to evaluate SVM, RF, and DST fusion. The DST fusion model achieved the best overall performance, showing that evidence-based fusion can improve IDS reliability [1].

Figure 8 shows the confusion matrix components used to calculate accuracy, precision, recall, and F1-score.

		Predicted Class	
		Benign	Attack
Actual Class	Benign	TN True Negative	FP False Positive
	Attack	FN False Negative	TP True Positive

Figure 8: Confusion Matrix Structure

1-14. Chapter Summary

This chapter presented the theoretical foundation of the project. It explained the role of IDS in cybersecurity and clarified the limitations of traditional detection methods. It also discussed machine learning-based IDS and introduced the two main classifiers used in the project: SVM and Random Forest. The chapter then explained ensemble learning and Dempster-Shafer Theory as a formal evidence fusion method. Finally, it presented the preprocessing techniques, LASSO feature selection, SMOTE balancing, CSE-CIC-IDS2018 dataset, and evaluation metrics. These concepts provide the necessary background for understanding the related studies discussed in the next chapter.

Chapter Two: Related Work

2-1. Introduction

This chapter reviews previous studies related to machine learning-based Intrusion Detection Systems, ensemble learning, hybrid IDS models, and Dempster-Shafer-based evidence fusion. The purpose of this chapter is not only to summarize previous work, but also to analyze how these studies support the current project and identify the research gap addressed by combining SVM and RF using DST.

The reviewed studies show that machine learning has become a major direction in IDS research because it can improve detection accuracy, reduce false alarms, and adapt to evolving attacks. However, many studies still rely on individual classifiers or simple ensemble mechanisms. Other studies use hybrid approaches but do not explicitly address uncertainty and conflict between classifier outputs. This creates a need for evidence-based fusion methods such as Dempster-Shafer Theory.

2-2. Development of Machine Learning-Based IDS

The development of IDS has moved from static rule-based systems toward intelligent data-driven systems. Traditional IDS methods depend mainly on known signatures and predefined rules. These approaches are effective against previously known threats but weak against new or modified attacks. Machine learning provides a more adaptive approach because models can learn from network traffic features and classify unseen data based on learned patterns [4].

Several machine learning algorithms have been applied to IDS, including Decision Trees, Support Vector Machines, Random Forest, Naive Bayes, Artificial Neural Networks, K-Nearest Neighbors, Gradient Boosting, and Deep Learning models. Each algorithm has advantages and limitations. For example, SVM can produce strong decision boundaries, but it may be computationally expensive for large datasets. Random Forest is robust and flexible, but its performance may depend on the quality of features and class distribution. Deep learning models can learn complex representations, but they often require large computational resources and may be less interpretable.

The reviewed literature shows that no single algorithm is universally best for all IDS scenarios. Network traffic is complex, attack patterns evolve over time, and datasets often suffer from imbalance. Therefore, researchers have increasingly focused on hybrid and ensemble approaches that combine multiple algorithms to improve detection reliability [4], [5], [7].

2-3. Ensemble Learning Approaches in IDS

Ensemble learning has been widely used to improve IDS performance. The main idea is to combine multiple models so that the strengths of one model can compensate for the weaknesses of another. In intrusion detection, ensemble methods can reduce false positives, improve detection rates, and provide more stable predictions.

Sharma and Shah studied the use of ensemble learning classifiers and hybrid feature selection for improving IDS performance. Their study compared different machine learning methods, including SVM, Random Forest, Decision Trees, and other models. The results showed that combining algorithms such as RF and SVM can improve performance compared with individual classifiers. Random Forest can provide robust feature representation, while SVM can

refine the decision boundary [4]. This directly supports the current project, which is also based on combining RF and SVM.

Dey and Bhakta presented an IDS model based on Random Forest and Support Vector Machine. According to the review table, their work showed that combining RF and SVM increases the efficiency of threat detection compared with using individual models. This study is closely related to the current project because it supports the compatibility between SVM's boundary-based classification and RF's generalization ability [5].

Shakeri, Raei, and Nejati proposed an approach that combines clustering and ensemble classifiers to improve IDS accuracy and reduce false alarm rates. Their work supports the idea that combining data handling techniques with multiple classifiers can improve IDS performance. This is relevant to the current project because preprocessing, feature selection, and balancing are used before classifier fusion [7].

These studies show that ensemble and hybrid learning methods are effective in IDS. However, many ensemble methods depend on voting, averaging, or stacking. These methods may improve accuracy, but they do not always provide a formal mechanism for handling uncertainty or conflict between classifier outputs. This limitation motivates the use of Dempster-Shafer Theory in the current project.

2-4. Evidence Fusion and Dempster-Shafer-Based Approaches

Evidence fusion is an important research direction in cybersecurity because detection decisions are often uncertain. Different classifiers may produce different outputs for the same traffic record, especially when the record is close to the decision boundary or when it contains ambiguous patterns. Dempster-Shafer Theory provides a mathematical framework for combining such evidence and explicitly managing uncertainty.

Dallali, Omrani, and Rhaimi proposed an evidence theory data fusion method for cyber-attack detection. Their approach combined decisions from SVM and ANN classifiers using Dempster-Shafer Theory. The study emphasized that cyberattack detection often occurs in uncertain environments, where data may be incomplete or imprecise. Their results showed that DST can improve the reliability of the final detection decision by combining evidence from independent classifiers [2]. This study provides an important theoretical foundation for the current project because it demonstrates the suitability of DST for cyberattack detection.

Galdames et al. presented a Dempster-Shafer fusion-based approach for malware detection. Although their work focuses on malware rather than network intrusion detection, it is highly relevant because it explains how DST can combine classifier outputs, construct mass functions, apply combination rules, and support uncertainty-aware cybersecurity decisions. Their experiments were conducted on more than 630,000 benign and malicious samples from multiple public datasets. The study showed that DST-based fusion can maintain low false positive rates and provide interpretable evidence combination [3]. This supports the use of DST in security systems where uncertainty and conflicting evidence are common.

The main paper of the current project applied DST directly to IDS by integrating SVM and RF classifiers. It used DST to convert the probabilistic outputs of the classifiers into belief assignments and then produce a final decision based on the highest belief score. The study

reported that DST fusion outperformed individual SVM and RF classifiers, achieving 97.84% accuracy and 96.17% F1-score on the CSE-CIC-IDS2018 dataset [1].

These studies show that DST is a promising method for cybersecurity applications. It is useful not only because it can combine multiple classifiers, but also because it can model uncertainty and conflict. This makes it more suitable than simple voting when classifier outputs are ambiguous.

2-5. Comparative Analysis of Related Studies

2-5-1. Summary Table of Related Studies

Table 1: Summary and Critical Analysis of Related Studies

No.	Study	Main Technique	Dataset / Scope	Main Contribution	Relevance to Current Project
1	Waheed et al., "Enhancing Intrusion Detection Systems by Integrating SVM and RF Using DST" [1]	SVM, RF, DST fusion, LASSO, SMOTE	CSE-CIC-IDS2018	Proposed a hybrid IDS that fuses SVM and RF outputs using DST and achieved better performance than individual classifiers.	This is the main paper on which the project is based.
2	Dallali et al., "Evidence Theory Data Fusion-Based Method for Cyber-Attack Detection" [2]	SVM, ANN, DST evidence fusion	DARPA / cyberattack detection	Combined classifier decisions using DST to improve detection under uncertainty.	Provides theoretical support for using DST in cyberattack detection.
3	Galdames et al., "A Dempster-Shafer Fusion-Based Approach for Malware Detection" [3]	DST-based ensemble fusion	BODMAS, DREBIN, AndroZoo, BMPD	Studied mass function construction and combination rules for DST-based malware detection.	Supports the explanation of DST, mass functions, and uncertainty handling in cybersecurity.
4	Sharma and Shah, "Ensemble Learning Classifiers and Hybrid Feature Selection for Enhancing IDS Performance" [4]	Ensemble learning, RF, SVM, feature selection	CICIDS2018	Showed that hybrid models using RF and SVM can improve IDS performance.	Supports the idea that combining RF and SVM is stronger than using a single classifier.
5	Liu, Gu, and Wang, "A Hybrid IDS Based on Scalable K-Means, Random	K-Means, RF, Deep Learning	NSL-KDD, CIC-IDS2017	Proposed a hybrid IDS combining clustering, RF, and deep learning.	Supports the general trend toward hybrid IDS architectures.

	Forest and Deep Learning" [6]				
6	Shakeri, Raci, and Nejati, "Improving Accuracy in IDS Using Classifier Ensemble and Clustering" [7]	Clustering and ensemble classifiers	NSL-KDD	Used clustering and ensemble learning to improve accuracy and reduce false alarms.	Supports the use of ensemble models and data handling before classification.
7	Dey and Bhakta, "A New RF and SVM-Based IDS Model in Networks" [5]	RF and SVM	Network intrusion data	Showed the usefulness of combining RF and SVM for intrusion detection.	Closely supports the classifier selection of the current project.
8	Yellepeddi, "AI-Powered IDS: Real-World Performance Analysis" [9]	AI-based IDS performance analysis	Practical IDS environments	Compared AI-based IDS with traditional systems in terms of accuracy and false alarms.	Supports the importance of AI-based IDS in practical cybersecurity environments.
9	Bhatt, "Improving IDS with Artificial Intelligence: A Review" [8]	ML, DL, Fuzzy Logic review	IDS techniques and applications	Reviewed AI techniques used in IDS and discussed their strengths and weaknesses.	Useful for explaining the development of IDS using artificial intelligence.

The reviewed studies confirm three major findings. First, machine learning has significantly improved IDS performance compared with traditional rule-based systems. Second, hybrid and ensemble models often outperform individual classifiers because they combine complementary learning capabilities. Third, uncertainty remains a major challenge in IDS, especially when classifiers produce conflicting decisions. Therefore, DST provides an important solution because it enables evidence-based fusion rather than simple aggregation.

2-6. Research Gap

Although previous studies have made significant progress in improving IDS performance, several gaps remain. Many IDS models still rely on a single classifier. Single models may be sensitive to class imbalance, noisy features, and complex attack patterns. SVM may produce strong decision boundaries but can struggle with large datasets. Random Forest can handle nonlinear relationships and feature diversity, but it may still produce uncertain decisions in ambiguous cases.

Another limitation is that many ensemble IDS models use simple fusion techniques such as voting or averaging. These methods combine decisions but do not explicitly represent uncertainty or conflict. For example, if SVM predicts an attack with moderate confidence while RF predicts benign traffic with high confidence, simple voting may not adequately represent the disagreement between the classifiers.

In addition, IDS datasets are often imbalanced. Benign traffic usually appears more frequently than attack traffic. Without proper balancing, models may become biased toward the

majority class. Feature redundancy is another issue because large traffic datasets may contain many features that do not contribute meaningfully to classification. Therefore, preprocessing, feature selection, and class balancing are necessary before training IDS models.

The reviewed literature shows that only a limited number of studies combine RF and SVM using Dempster-Shafer Theory for intrusion detection. This gap is important because RF and SVM provide complementary classification behavior, while DST provides a formal method for fusing their outputs under uncertainty. The current project addresses this gap by presenting a hybrid IDS framework that integrates SVM and RF through DST and evaluates it on the CSE-CIC-IDS2018 dataset [1].

2-7. Position of the Current Project

The current project is positioned within the field of intelligent hybrid intrusion detection systems. It builds on previous research that shows the effectiveness of machine learning and ensemble models in IDS. However, it focuses specifically on improving decision reliability through evidence fusion.

The proposed approach differs from traditional single-classifier IDS models because it combines two classifiers with different strengths. SVM contributes strong boundary-based classification, while RF contributes ensemble-based robustness and feature diversity. The approach also differs from simple ensemble methods because it uses Dempster-Shafer Theory to combine classifier outputs in a formal uncertainty-aware manner.

The project also incorporates important preprocessing steps. The CSE-CIC-IDS2018 dataset is cleaned, standardized, balanced using SMOTE, and reduced using LASSO feature selection. These steps improve the quality of training data and reduce computational complexity. After classification, DST fusion produces the final decision by combining belief assignments from SVM and RF.

Based on the related work, the current project can be considered a hybrid IDS model that addresses three major challenges: classifier limitation, data imbalance, and decision uncertainty. This makes the approach suitable for complex cybersecurity environments where attacks are diverse and traffic behavior can be ambiguous.

2-8. Chapter Summary

This chapter reviewed studies related to machine learning-based IDS, ensemble learning, hybrid detection models, and Dempster-Shafer evidence fusion. The reviewed literature showed that ML-based IDS models improve detection compared with traditional systems, while ensemble and hybrid models provide stronger performance than individual classifiers. Studies based on DST demonstrated the importance of evidence fusion in uncertain cybersecurity environments. The chapter also identified the research gap addressed by the current project: the need for a hybrid IDS that combines SVM and RF while explicitly handling uncertainty through Dempster-Shafer Theory. This analysis provides the basis for presenting the proposed methodology in the next chapter.

Chapter Three: Proposed Methodology

3-1. Introduction

This chapter presents the proposed methodology used to build the hybrid Intrusion Detection System. The methodology is based on integrating two machine learning classifiers, namely Support Vector Machine and Random Forest, using Dempster-Shafer Theory as a decision fusion mechanism. The main purpose of the proposed methodology is to improve intrusion detection reliability by combining the strengths of both classifiers and reducing the uncertainty that may appear when a single classifier is used.

The proposed approach follows a structured pipeline that begins with preparing the CSE-CIC-IDS2018 dataset, cleaning and standardizing the data, selecting the most relevant features using LASSO, balancing the classes using SMOTE, training SVM and Random Forest classifiers, and finally combining their outputs through Dempster-Shafer Theory. This methodology is suitable for intrusion detection because network traffic data is usually large, imbalanced, and affected by uncertainty in distinguishing between benign and malicious behavior [1].

3-2. General Architecture of the Proposed IDS

3-2-1. Main Stages of the Architecture

The proposed IDS architecture consists of several sequential stages. First, the network traffic dataset is collected and prepared. Then, preprocessing operations are applied to remove invalid values, transform labels, and standardize numerical features. After that, LASSO feature selection is used to reduce the dimensionality of the dataset and keep only the most discriminative traffic features. Since the dataset contains an imbalance between benign and attack samples, SMOTE is applied to improve the representation of the minority class.

After preparing the data, two classifiers are trained independently: Support Vector Machine and Random Forest. Each classifier produces a prediction output for each network traffic record. Instead of selecting one model or using simple voting, the proposed system converts the outputs of both classifiers into belief assignments. These belief values are then combined using Dempster-Shafer Theory to produce the final classification decision.

The architecture is designed to address three main challenges in IDS. The first challenge is high dimensionality, which is handled using LASSO feature selection. The second challenge is class imbalance, which is handled using SMOTE. The third challenge is decision uncertainty, which is handled using Dempster-Shafer-based fusion [1].

Figure 9 presents the full architecture of the proposed IDS from dataset preparation to the final DST-based decision.

General Architecture of the Proposed Hybrid IDS

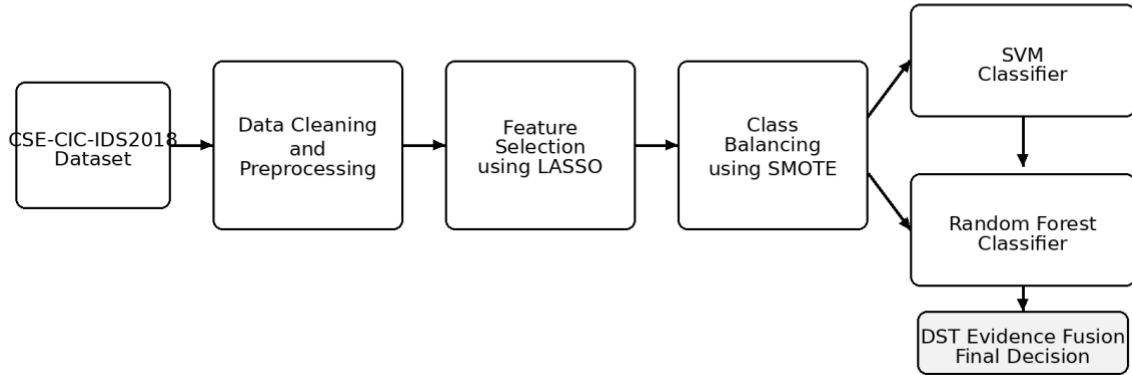


Figure 9: General Architecture of the Proposed Hybrid IDS

3-3. Dataset Preparation

The proposed system uses the CSE-CIC-IDS2018 dataset. This dataset is widely used in intrusion detection research because it contains realistic network traffic flows, including benign traffic and several types of attacks. The dataset includes multiple attack scenarios, such as DDoS-HOIC, LOIC-UDP, SSH brute-force, SQL injection, and infiltration. It contains millions of labeled records and a large number of numerical flow features, which makes it suitable for evaluating machine learning-based IDS models [1].

In the proposed methodology, the dataset is treated as a binary classification problem. Benign traffic is assigned to class 0, while all attack traffic categories are assigned to class 1. This binary formulation reflects a practical IDS scenario in which the system must decide whether a traffic flow is normal or malicious. Although multi-class classification can provide more detailed attack categorization, binary classification is useful when the main objective is early detection of suspicious activity.

The dataset is divided into training and testing subsets. The training subset is used to train the classifiers, while the testing subset is used to evaluate the final performance. Stratified sampling is used to preserve the original class distribution in both subsets. In the main methodology, the data partitioning ratio is 80% for training and 20% for testing with a fixed random seed to support reproducibility [1].

3-4. Data Preprocessing

Data preprocessing is a necessary step before training the machine learning models. Raw network traffic datasets may contain missing values, infinite values, redundant rows, non-informative identifiers, and features with different numerical scales. These issues can negatively affect classifier performance and lead to unreliable detection results.

The preprocessing stage includes removing redundant header rows and eliminating non-informative identifiers such as IP addresses and timestamps. These fields are not used directly for classification because they may not represent generalizable attack behavior. Missing values and infinite values are handled to ensure that all features are valid numerical inputs. Label encoding is also applied to convert class labels into numerical form.

Feature standardization is applied using StandardScaler. This step transforms numerical features so that they have a consistent scale. Standardization is particularly important for SVM because the algorithm depends on distances and margins between samples. If features are not scaled, features with large numerical ranges may dominate the learning process. Standardization also helps create more consistent input for both SVM and Random Forest classifiers [1].

3-5. Feature Selection Using LASSO

After preprocessing, the dataset may still contain many features, some of which are redundant or weakly related to the classification task. To reduce dimensionality and improve computational efficiency, the proposed methodology uses LASSO feature selection.

LASSO stands for Least Absolute Shrinkage and Selection Operator. It uses L1 regularization to shrink the coefficients of less important features toward zero. Features with zero or near-zero coefficients are considered less informative and can be removed. This makes LASSO useful for selecting the most relevant traffic features while reducing the complexity of the IDS model.

In the main study, the original feature set contained 78 numerical features. After applying LASSO, 46 discriminative features were selected. These features included packet length statistics, flow inter-arrival time, flag counts, header lengths, active and idle flow information, and throughput-related measurements [1]. These selected features are important because they describe statistical, temporal, and protocol-level behavior of network traffic.

Using LASSO provides several advantages. It reduces training time, improves model interpretability, removes redundant information, and helps the classifiers focus on the most meaningful indicators of malicious behavior. This is especially important for large-scale IDS datasets because training on unnecessary features can increase computational cost without improving detection accuracy.

3-6. Class Balancing Using SMOTE

Class imbalance is one of the main challenges in intrusion detection datasets. In real network environments, benign traffic is usually more common than attack traffic. If a model is trained on imbalanced data, it may become biased toward the majority class and fail to detect minority attack samples effectively.

To address this problem, the proposed methodology applies the Synthetic Minority Oversampling Technique. SMOTE generates synthetic samples for the minority class by creating new examples between existing minority samples. Unlike simple duplication, SMOTE increases minority class representation in a way that helps the model learn more general decision boundaries.

In the main methodology, SMOTE is applied only to the training data. This is important because applying oversampling to the testing data would produce unrealistic evaluation results. After applying SMOTE, the training data becomes more balanced, which improves the ability of SVM and Random Forest to identify attack traffic [1].

Class balancing is important not only for individual classifiers but also for the final fusion process. If the base classifiers are trained on imbalanced data, their probability outputs may be

biased. Since Dempster-Shafer fusion depends on classifier outputs, improving the quality of these outputs improves the reliability of the final belief-based decision.

3-7. Support Vector Machine Classification

Support Vector Machine is used as one of the two base classifiers in the proposed IDS. SVM aims to find the optimal hyperplane that separates benign and malicious traffic samples. The optimal hyperplane is selected by maximizing the margin between classes. This makes SVM effective in classification problems where a clear boundary can be identified between different classes.

In the proposed methodology, SVM is used because it performs well with high-dimensional data and can produce strong decision boundaries. These characteristics are useful in IDS because network traffic records contain many numerical features. However, SVM can be computationally expensive when applied to very large datasets. Therefore, the main study used GPU-accelerated SVM through RAPIDS cuML to handle large-scale training more efficiently [1].

The SVM model is trained on the selected and balanced feature set. Hyperparameter tuning is performed using grid search. The main hyperparameters include the regularization parameter C , the kernel type, and the gamma parameter. The tested values include $C \in \{0.1, 1, 10\}$, kernel $\in \{\text{linear}, \text{rbf}\}$, and gamma $\in \{\text{scale}, 0.01, 0.001\}$ [1].

The output of SVM is not used alone as the final IDS decision. Instead, its probabilistic output is later transformed into a belief assignment and combined with the Random Forest output using Dempster-Shafer Theory.

3-8. Random Forest Classification

Random Forest is used as the second base classifier in the proposed IDS. RF is an ensemble algorithm that builds multiple decision trees and combines their predictions. Each tree is trained on a subset of the data, and the final prediction is obtained by aggregating the decisions of all trees.

RF is suitable for intrusion detection because it can capture nonlinear relationships between features and handle complex traffic patterns. It is also robust against overfitting compared with a single decision tree because the final decision is based on multiple trees rather than one model. In network traffic analysis, different attacks may be reflected in different feature groups, and RF can use its ensemble structure to learn these diverse patterns.

The Random Forest model is trained using the same selected and balanced dataset used for SVM. Hyperparameter tuning is performed through grid search. The tested hyperparameters include the number of trees, maximum tree depth, and minimum samples required for splitting. The search space includes $n_estimators \in \{100, 200, 300\}$, $max_depth \in \{10, 20, \text{None}\}$, and $min_samples_split \in \{2, 5, 10\}$ [1].

Like SVM, RF produces probability outputs for the benign and attack classes. These outputs are later used as evidence in the Dempster-Shafer fusion stage.

3-9. Dempster-Shafer-Based Decision Fusion

The final stage of the proposed methodology is decision fusion using Dempster-Shafer Theory. The purpose of this stage is to combine the outputs of SVM and Random Forest in a formal way that accounts for uncertainty and conflict between classifiers.

3-9-1. Frame of Discernment

The frame of discernment is defined as:

$$\Theta = \{\text{Benign}, \text{Attack}\} \quad (8)$$

3-9-2. Conversion to Belief Assignments

For each traffic record, each classifier produces a probability value for the attack class. This probability is converted into a Basic Belief Assignment. For a classifier output probability p attack, the belief masses can be defined as:

$$m(\{0\}) = 1 - p_{\text{attack}} \quad (9)$$

$$m(\{1\}) = p_{\text{attack}} \quad (10)$$

$$m(\Theta) = 0 \quad (11)$$

where $\{0\}$ represents benign traffic, $\{1\}$ represents attack traffic, and Θ represents uncertainty. In this simplified formulation, the classifier assigns belief to the benign and attack hypotheses based on its probability output.

3-9-3. Conflict Coefficient

After obtaining belief assignments from SVM and Random Forest, the conflict between the two evidence sources is calculated. The conflict coefficient K is defined as:

$$K = m_A(\{0\})m_B(\{1\}) + m_A(\{1\})m_B(\{0\}) \quad (12)$$

where m_A and m_B represent the belief assignments of the two classifiers. A higher value of K indicates stronger disagreement between SVM and Random Forest.

The combined belief for each hypothesis is calculated using Dempster's rule of combination. For the benign class, the combined belief can be expressed as:

$$m_{AB}(\{0\}) = [m_A(\{0\})m_B(\{0\}) + m_A(\{0\})m_B(\Theta) + m_A(\Theta)m_B(\{0\})] / (1 - K) \quad (13)$$

For the attack class, the combined belief can be expressed as:

$$m_{AB}(\{1\}) = [m_A(\{1\})m_B(\{1\}) + m_A(\{1\})m_B(\Theta) + m_A(\Theta)m_B(\{1\})] / (1 - K) \quad (14)$$

3-9-4. Final Decision Rule

The final decision is selected based on the class with the highest combined belief value:

$$\hat{y} = 0, \text{ if } m_{AB}(\{0\}) \geq m_{AB}(\{1\}); \text{ otherwise, } \hat{y} = 1 \quad (15)$$

This fusion mechanism is more structured than simple voting because it considers both agreement and conflict between classifiers. When SVM and RF agree, the fusion process

strengthens the final decision. When they disagree, the conflict is handled through the normalization factor in Dempster's rule. This makes the approach suitable for uncertain IDS environments where traffic behavior may be ambiguous [1], [2], [3].

3-10. Proposed Algorithm

The proposed IDS methodology can be summarized as follows:

3-10-1. Algorithm Specification

Algorithm 1: Hybrid IDS Using SVM, Random Forest, and Dempster-Shafer Theory

Input: CSE-CIC-IDS2018 network traffic dataset Output: Final classification label: Benign or Attack

1. Load the CSE-CIC-IDS2018 dataset.
2. Remove redundant rows, non-informative identifiers, and invalid values.
3. Encode the class labels into binary form: benign as 0 and attack as 1.
4. Standardize the numerical features using StandardScaler.
5. Apply LASSO feature selection to select the most discriminative features.
6. Split the dataset into training and testing subsets using stratified sampling.
7. Apply SMOTE to the training subset to balance benign and attack classes.
8. Train the SVM classifier using the selected and balanced features.
9. Train the Random Forest classifier using the selected and balanced features.
10. Generate probability outputs from both classifiers.
11. Convert classifier probabilities into Basic Belief Assignments.
12. Calculate the conflict coefficient between SVM and Random Forest evidence.
13. Apply Dempster's rule of combination to fuse the belief assignments.
14. Select the final class according to the highest combined belief value.
15. Evaluate the final model using accuracy, precision, recall, F1-score, and confusion matrix analysis.

This algorithm shows how the proposed system moves from raw data to final IDS decision. The method combines preprocessing, feature selection, balancing, classification, and evidence fusion into one complete intrusion detection pipeline.

3-10-2. Algorithm Flowchart

Figure 10 summarizes the complete implementation flow of the proposed hybrid IDS algorithm.

Complete Proposed Algorithm Flowchart

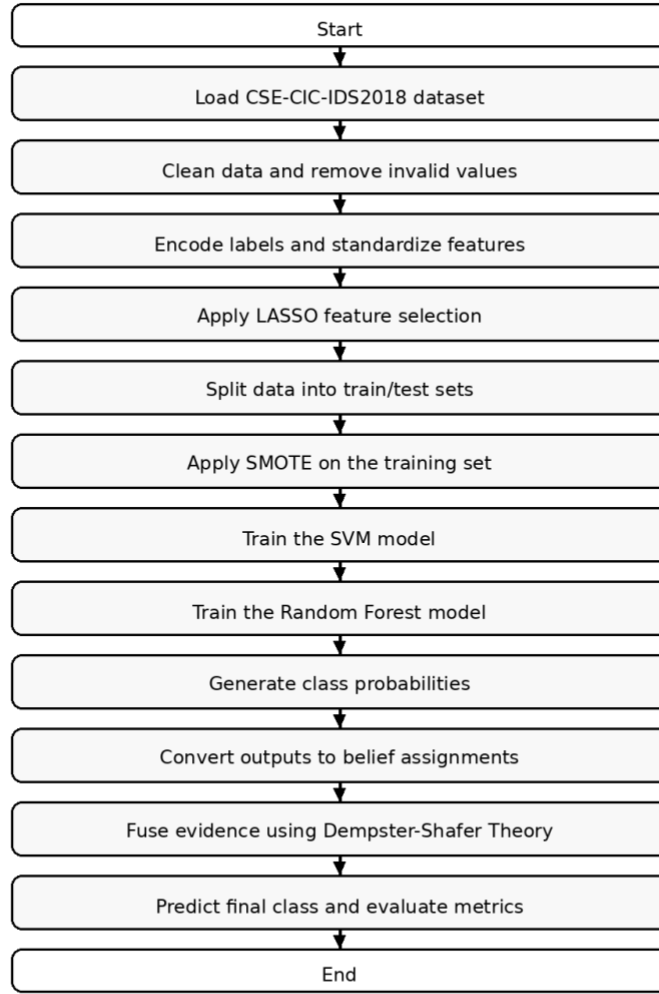


Figure 10: Complete Proposed Algorithm Flowchart

3-11. Experimental and Implementation Configuration

Table 2: Experimental and Implementation Configuration of the Proposed IDS [1]

Item	Specification
Dataset	CSE-CIC-IDS2018 network traffic dataset
Classification Type	Binary classification: benign and attack
Data Partition	80% training and 20% testing
Sampling Method	Stratified sampling
Random Seed	42
Preprocessing	Data cleaning, label encoding, missing value handling, standardization
Standardization Tool	StandardScaler

Class Balancing	SMOTE
Feature Selection	LASSO
Number of Original Features	78 numerical features
Number of Selected Features	46 selected features
SVM Hyperparameters	$C \in \{0.1, 1, 10\}$; kernel $\in \{\text{linear}, \text{rbf}\}$; gamma $\in \{\text{scale}, 0.01, 0.001\}$
Random Forest Hyperparameters	n_estimators $\in \{100, 200, 300\}$; max_depth $\in \{10, 20, \text{None}\}$; min_samples_split $\in \{2, 5, 10\}$
Fusion Method	Dempster-Shafer Theory
Decision Criterion	Class with maximum belief score after fusion
Evaluation Metrics	Accuracy, precision, recall, F1-score, confusion matrix
Main Libraries	scikit-learn, cuML, imbalanced-learn
Development Language	Python
Development Environment	Visual Studio Code
Implementation Plan	The same algorithmic settings of the main study will be adopted initially. Hardware-specific settings will be updated according to the available machine during implementation.
Reference Hardware in Main Study	Intel Core i7 processor, NVIDIA A100 GPU, 16 GB RAM

The experimental configuration provides a reproducible structure for implementing and evaluating the proposed IDS. It also clarifies the role of each component in the methodology. The preprocessing and feature selection stages prepare the data, the SVM and RF models perform classification, and DST combines the classifier outputs into a final decision. In the project implementation, Python will be used as the programming language inside Visual Studio Code. The initial implementation will follow the same algorithmic settings reported in the main study, while local hardware specifications can be documented later in the practical implementation chapter.

3-12. Chapter Summary

This chapter presented the proposed methodology for the hybrid intrusion detection system. The proposed approach starts with preparing the CSE-CIC-IDS2018 dataset and applying preprocessing operations to ensure clean and consistent input data. LASSO is then used to select the most important traffic features, while SMOTE is applied to handle class imbalance. After that, SVM and Random Forest classifiers are trained independently. Their probabilistic outputs are converted into belief assignments and fused using Dempster-Shafer Theory. The final decision is selected according to the highest combined belief value. This methodology provides a structured IDS framework that combines the strengths of SVM and RF while addressing uncertainty through evidence-based fusion.

Tables 3 and 4 provide additional clarification of the algorithmic components and the main input-output flow of the proposed IDS. They are added to support the explanation of the methodology and make the role of each algorithmic stage clearer.

Table 3: Algorithmic Components and Their Roles in the Proposed IDS

Component	Function in the Methodology	Reason for Use	Output
Data Preprocessing	Cleans and prepares raw	Removes invalid values	Clean and standardized

	network traffic data.	and creates consistent numerical inputs.	dataset.
LASSO Feature Selection	Selects the most discriminative traffic features.	Reduces dimensionality and removes redundant predictors.	Reduced feature set with 46 selected features.
SMOTE	Balances the minority attack class in the training data.	Reduces classifier bias toward the benign majority class.	Balanced training subset.
Support Vector Machine	Builds a margin-based boundary between benign and attack samples.	Provides strong classification boundaries for high-dimensional data.	SVM probability output.
Random Forest	Combines multiple decision trees to classify traffic.	Captures nonlinear relationships and improves robustness.	RF probability output.
Dempster-Shafer Theory	Fuses SVM and RF outputs using belief assignments.	Handles uncertainty and conflict between classifier decisions.	Final benign or attack decision.
Evaluation Metrics	Measures the quality of the IDS output.	Provides a balanced view of accuracy, false alarms, and missed attacks.	Accuracy, precision, recall, F1-score, and confusion matrix.

Table 4: Inputs and Outputs of the Main Processing Stages

Stage	Input	Main Operation	Output
Dataset Preparation	CSE-CIC-IDS2018 files	Merge and define binary labels.	Benign/attack traffic records.
Preprocessing	Raw traffic records	Clean values, encode labels, and standardize features.	Model-ready numerical data.
Feature Selection	Standardized features	Apply LASSO to retain informative predictors.	Selected feature matrix.
Class Balancing	Training subset	Apply SMOTE to minority attack samples.	Balanced training data.
SVM Classification	Selected balanced features	Train and tune the SVM classifier.	Class probability estimates.
RF Classification	Selected balanced features	Train and tune the Random Forest classifier.	Class probability estimates.
DST Fusion	SVM and RF probabilities	Convert to belief assignments and combine evidence.	Final predicted class.
Evaluation	Predicted and true labels	Compute confusion matrix and derived metrics.	Performance report.

Chapter Four: Practical Implementation

4-1. Introduction

This chapter presents the practical implementation of the proposed hybrid intrusion detection system. It focuses on how the methodology described in Chapter Three was translated into executable Python code. The implementation includes dataset loading, preprocessing, class balancing, feature selection, model training, Dempster-Shafer fusion, and final evaluation. The purpose of this chapter is to document the practical workflow and the main code sections used to implement the system.

The screenshots included in this chapter are divided according to their functional role in the implementation. Each code segment is presented separately and explained directly below its figure so that the relationship between the implementation step and the proposed methodology remains clear.

4-2. Practical Work Environment

The implementation was carried out using a local development environment suitable for Python-based machine learning experiments. Table 5 summarizes the hardware and software environment used in the project.

Table 5: Practical Work Environment and Dataset

Item	Specification
Device	Asus Zenbook
RAM	16 GB
Processor	Intel Core i7 Evo
Operating System	Windows 11
Programming Language	Python
Development Environment	Visual Studio Code
Dataset	CSE-CIC-IDS2018

The CSE-CIC-IDS2018 dataset is used as the practical dataset for the implementation. The local CSV file shown in the code represents the prepared dataset file used during the experiment.

4-3. Dataset Used in the Practical Implementation

The practical implementation in this project uses the CSE-CIC-IDS2018 dataset as the main input source for training and testing the intrusion detection models. This dataset is suitable for the implemented system because it contains labeled network traffic flows representing both benign traffic and multiple attack categories. In the practical code, the dataset is loaded from a local CSV file named `balanced_dataset1.csv`, which is processed as the working dataset for the experiment. Before running the implementation pipeline, the working dataset was manually balanced to ensure an equal representation of benign and attack records. After this manual balancing step, the dataset used in the practical implementation contained 1,559,318 samples in total.

For implementation purposes, the dataset is converted into a binary classification problem. Benign traffic is assigned to class 0, while all attack categories are assigned to class 1. This binary representation is consistent with the objective of the implemented IDS, which is to determine whether a network flow is normal or malicious. The dataset initially contains 78 numerical features, and after applying LASSO feature selection, the feature space is reduced to 46 selected features.

The dataset is also important for validating the complete processing pipeline. It is used to demonstrate data cleaning, label conversion, train-test splitting, class-balancing verification, feature selection, scaling, classifier training, and Dempster-Shafer-based fusion. Therefore, the dataset is not only a data source, but also a central part of the practical implementation environment.

4-4. Dataset Loading and Initial Preparation

4-4-1. Library Import and Dataset Loading

Figure 11 shows the first part of the implementation. This part imports the required Python libraries and defines the dataset file path before loading the data into a pandas DataFrame.

```
1 import numpy as np
2 import pandas as pd
3 import warnings
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.metrics import (
7     accuracy_score, precision_score, recall_score, f1_score,
8     confusion_matrix, classification_report
9 )
10 from sklearn.svm import SVC
11 from sklearn.ensemble import RandomForestClassifier
12 from sklearn.calibration import CalibratedClassifierCV
13 from sklearn.linear_model import LassoCV
14 from sklearn.feature_selection import SelectFromModel
15 from imblearn.over_sampling import SMOTE
16
17 warnings.filterwarnings("ignore")
18
19 files = [r"C:\Users\ahmed\Downloads\balanced_dataset1.csv"]
20
21 print("File Loading...")
22 df_list = [pd.read_csv(f) for f in files]
23 df = pd.concat(df_list, ignore_index=True)
24
```

Figure 11: Library Import and Dataset Loading Code

This code segment imports NumPy and pandas for numerical and tabular data processing. It also imports the main machine learning tools used later in the implementation, including train-test splitting, standardization, evaluation metrics, SVM, Random Forest, probability calibration, LASSO-based selection, and SMOTE. After suppressing warning messages, the code defines the CSV dataset path, loads the file, concatenates it into a single DataFrame, and prints the unique label values to inspect the class names before cleaning.

4-4-2. Label Cleaning, Feature Preparation, and Train-Test Split

Figure 12 presents the cleaning and splitting stage. This part prepares the target labels, removes non-informative columns, converts features to numeric values, handles invalid values, and divides the dataset into training and testing subsets.

```

26 print(df["Label"].unique())
27
28
29 df.columns = df.columns.str.strip()
30 df["Label"] = df["Label"].astype(str).str.strip()
31 df = df[df["Label"] != "Label"]
32
33
34
35 df["Label_bin"] = df["Label"].apply(lambda x: 0 if x == "Benign" else 1)
36
37 drop_cols = ["Flow ID", "Src IP", "Src Port", "Dst IP", "Timestamp", "SimillarHTTP", "Inbound"]
38 X = df.drop(columns=drop_cols + ["Label", "Label_bin"], errors="ignore")
39 y = df["Label_bin"]
40
41 X = X.apply(pd.to_numeric, errors='coerce')
42 X.replace([np.inf, -np.inf], np.nan, inplace=True)
43 X.fillna(0, inplace=True)
44 X = X.clip(lower=-1e15, upper=1e15)
45
46
47 X_train, X_test, y_train, y_test = train_test_split(
48     X, y, test_size=0.2, random_state=42, stratify=y
49 )

```

Figure 12: Label Cleaning, Feature Preparation, and Train-Test Split Code

The column names and label values are first stripped from unnecessary spaces to avoid errors caused by inconsistent formatting. The label column is converted into a binary target named `Label_bin`, where benign traffic is represented by 0 and attack traffic is represented by 1. Non-generalizable identifiers such as Flow ID, source and destination IP addresses, ports, and timestamps are removed because they do not represent stable behavioral features. The feature matrix is then converted to numeric form, missing and infinite values are replaced, extreme values are clipped, and the data is split into 80% training and 20% testing using stratification to preserve the class distribution.

4-5. Data Balancing, Feature Selection, and Scaling

4-5-1. Class Balancing Using SMOTE

Figure 13 shows the SMOTE stage used to balance the training subset after the train-test split.

```

51 print(f"Train shape: {X_train.shape}")
52 print(f"Test shape : {X_test.shape}")
53
54 print("Before SMOTE:", pd.Series(y_train).value_counts().to_dict())
55
56 smote = SMOTE(random_state=42)
57 X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)
58
59 print("After SMOTE :", pd.Series(y_train_sm).value_counts().to_dict())
60

```

Figure 13: SMOTE Class Balancing Code

The code first prints the shape of the training and testing sets and displays the class distribution before balancing. SMOTE is then initialized with `random_state=42` to ensure reproducibility. The `fit_resample` function is applied only to the training data, generating synthetic samples for the minority class. This prevents the classifier from being biased toward the majority class while keeping the testing data unchanged and realistic.

4-5-2. LASSO Feature Selection

Figure 14 presents the feature selection step based on LASSO and SelectFromModel.

```
62 print("Lasso full training shape:", X_train_sm.shape)
63 print(pd.Series(y_train_sm).value_counts())
64
65 lasso = LassoCV(cv=3, random_state=42, max_iter=3000)
66 lasso.fit(X_train_sm, y_train_sm)
67 selector = SelectFromModel(lasso, prefit=True, max_features=46, threshold=-np.inf)
68
69 X_train_sel = selector.transform(X_train_sm)
70 X_test_sel = selector.transform(X_test.values)
71
72 print("Selected full training shape:", X_train_sel.shape)
73 print(pd.Series(y_train_sm).value_counts())
```

Figure 14: LASSO Feature Selection Code

After balancing, the code prints the new training shape and class distribution to confirm the effect of SMOTE. LassoCV is then trained on the balanced training set using cross-validation. SelectFromModel uses the trained LASSO model to retain the most informative features, with max_features=46 to match the methodology adopted in the project. The same selected feature mask is then applied to both training and testing data so that the models are trained and evaluated on the same feature subset.

4-5-3. Feature Scaling

Figure 15 shows the standardization step applied after feature selection.

```
75 scaler = StandardScaler()
76 X_train_scaled = scaler.fit_transform(X_train_sel)
77 X_test_scaled = scaler.transform(X_test_sel)
78
79
```

Figure 15: Feature Scaling Code

The selected features are standardized using StandardScaler. The scaler is fitted on the selected training features and then applied to both the training and testing feature sets. This step is particularly important for SVM because the model depends on distances and margins; unscaled features with large ranges could dominate the classification boundary. Using the same scaler for training and testing also prevents data leakage.

4-6. Model Training Implementation

4-6-1. Support Vector Machine Training

Figure 16 shows the implementation of the SVM classifier and its probability calibration.

```

80 print("Models are being trained...\n")
81 from sklearn.svm import LinearSVC
82
83 base_svm = LinearSVC(
84     C=1.0,
85     class_weight="balanced",
86     random_state=42,
87     max_iter=5000
88 )
89
90 svm_model = CalibratedClassifierCV(base_svm, method="sigmoid", cv=3)
91 svm_model.fit(X_train_scaled, y_train_sm)

```

Figure 16: SVM Training and Probability Calibration Code

A LinearSVC model is used as the base SVM classifier in this implementation. LinearSVC was selected instead of a general kernel-based SVM because the working dataset is large, and standard kernel SVM models can require significantly more memory and processing time when trained on a large number of network traffic samples. Therefore, LinearSVC was a more practical and computationally efficient choice for the available local environment. The parameter `C=1.0` controls the regularization strength, `class_weight="balanced"` helps reduce the influence of class imbalance, and `random_state=42` supports reproducibility. Since LinearSVC does not directly provide probability outputs, `CalibratedClassifierCV` is applied with the sigmoid method and three-fold cross-validation. This calibration step is essential because the later Dempster-Shafer fusion stage requires probabilistic evidence from both classifiers.

4-6-2. Random Forest Training and Probability Outputs

Figure 17 shows the implementation of the Random Forest classifier and the extraction of probability outputs from both base models.

```

92
93 rf_model = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
94 rf_model.fit(X_train_scaled, y_train_sm)
95
96 svm_prob = svm_model.predict_proba(X_test_scaled)
97 rf_prob = rf_model.predict_proba(X_test_scaled)

```

Figure 17: Random Forest Training and Probability Output Code

The Random Forest classifier is trained using 200 decision trees and `random_state=42`. The `n_jobs=-1` setting allows the model to use available processor resources for faster training. After both classifiers are fitted, `predict_proba` is used to obtain class probability estimates for the testing set. These probability values represent the classifier evidence that will later be converted into belief assignments for Dempster-Shafer fusion.

4-7. Evaluation and Dempster-Shafer Fusion Implementation

4-7-1. Evaluation Function

Figure 18 presents the evaluation function used to compute the main performance metrics.


```

100 def evaluate_model(name, y_true, y_pred):
101     acc = accuracy_score(y_true, y_pred)
102     prec = precision_score(y_true, y_pred, zero_division=0)
103     rec = recall_score(y_true, y_pred, zero_division=0)
104     f1 = f1_score(y_true, y_pred, zero_division=0)
105     cm = confusion_matrix(y_true, y_pred)
106
107     print(f"\n{name}")
108     print("-" * 50)
109     print("Accuracy :", acc)
110     print("Precision:", prec)
111     print("Recall   :", rec)
112     print("F1-score :", f1)
113     print("Confusion Matrix:\n", cm)
114     print("Classification Report:\n", classification_report(y_true, y_pred, zero_division=0))
115
116     return {
117         "Model": name, "Accuracy": acc, "Precision": prec, "Recall": rec, "F1-score": f1
118     }

```

Figure 18: Model Evaluation Function Code

The `evaluate_model` function receives the model name, true labels, and predicted labels. It calculates accuracy, precision, recall, F1-score, and the confusion matrix. It also prints a classification report to provide class-level evaluation details. Finally, the function returns the core metrics in dictionary form, which makes it easier to build a summary table comparing SVM, Random Forest, and the hybrid DST-based model.

4-7-2. Conversion to Belief Assignments

Figure 19 shows how classifier probabilities are converted into belief values suitable for Dempster-Shafer fusion.

```

120
121 def belief_from_probs(probs):
122     p_attack = probs[:, 1]
123     p_benign = probs[:, 0]
124     uncertainty = 1 - np.abs(p_attack - p_benign)
125     m_attack = p_attack * (1 - uncertainty * 0.1)
126     m_benign = p_benign * (1 - uncertainty * 0.1)
127     m_u = 1 - (m_attack + m_benign)
128     return m_benign, m_attack, m_u

```

Figure 19: Belief Assignment Construction Code

The `belief_from_probs` function extracts the probability of the attack class and the probability of the benign class from each classifier output. It also calculates an uncertainty value based on the difference between the two probabilities. The attack and benign masses are then adjusted according to this uncertainty, while the remaining mass is assigned to uncertainty. This creates three belief components for each classifier: benign belief, attack belief, and uncertainty belief.

4-7-3. Dempster-Shafer Combination and Final Decision

Figure 20 presents the implementation of Dempster-Shafer combination and the final decision rule.


```

129
130 def dempster_combination(m1_b, m1_a, m1_u, m2_b, m2_a, m2_u):
131     K = (m1_b * m2_a) + (m1_a * m2_b)
132     denominator = 1 - K
133     denominator = np.where(denominator <= 0, 1e-9, denominator)
134     f_b = (m1_b * m2_b + m1_b * m2_u + m1_u * m2_b) / denominator
135     f_a = (m1_a * m2_a + m1_a * m2_u + m1_u * m2_a) / denominator
136     f_u = (m1_u * m2_u) / denominator
137     total = f_b + f_a + f_u
138     return f_b/total, f_a/total, f_u/total
139
140 svm_mb, svm_ma, svm_mu = belief_from_probs(svm_prob)
141 rf_mb, rf_ma, rf_mu = belief_from_probs(rf_prob)
142
143 dst_b, dst_a, dst_u = dempster_combination(svm_mb, svm_ma, svm_mu, rf_mb, rf_ma, rf_mu)
144 dst_pred_base = (dst_a > dst_b).astype(int)
145
146 dst_pred = np.where(rf_prob[:, 1] >= 0.80, 1,
147                    np.where(rf_prob[:, 1] <= 0.20, 0, dst_pred_base)).astype(int)
148

```

Figure 20: Dempster-Shafer Combination and Final Decision Code

The dempster_combination function calculates the conflict coefficient K between the SVM and Random Forest belief assignments. The denominator $1 - K$ is protected from becoming zero by using a small numerical value when necessary. The function then computes the fused benign, attack, and uncertainty masses and normalizes them so that they form a valid belief distribution. After the SVM and RF probabilities are converted into belief assignments, the fused decision is produced by comparing the attack and benign masses. Additional probability thresholds are applied to the Random Forest output to strengthen very confident RF decisions before producing the final hybrid prediction.

4-8. Final Evaluation and Debugging Outputs

Figure 21 shows the final evaluation and debugging stage of the implementation.

```

150 svm_results = evaluate_model("SVM (Individual)", y_test, svm_model.predict(X_test_scaled))
151 rf_results = evaluate_model("Random Forest (Individual)", y_test, rf_model.predict(X_test_scaled))
152 dst_results = evaluate_model("SVM + RF + DST (Hybrid System)", y_test, dst_pred)
153
154 print("\n--- Summary Table ---")
155 results_summary = pd.DataFrame([svm_results, rf_results, dst_results])
156 print(results_summary[["Model", "Accuracy", "F1-score", "Precision", "Recall"]])
157
158 print("\n--- Fusion Debug (Top 10) ---")
159 fusion_debug = pd.DataFrame({
160     "svm_attack": svm_ma[:10], "rf_attack": rf_ma[:10], "dst_attack": dst_a[:10],
161     "final_pred": dst_pred[:10], "true_label": y_test.iloc[:10].values
162 })
163 print(fusion_debug)
164
165 rf_wrong = (rf_model.predict(X_test_scaled) != y_test.values)
166 dst_wrong = (dst_pred != y_test.values)
167 print("Error analysis...\n")
168 print(f"RF wrong: {rf_wrong.sum()}")
169 print(f"DST wrong: {dst_wrong.sum()}")
170 print(f"DST fixes RF errors: {(rf_wrong & ~dst_wrong).sum()}")

```

Figure 21: Final Evaluation, Summary Table, and Fusion Debugging Code

The final part of the code evaluates the individual SVM model, the individual Random Forest model, and the hybrid SVM + RF + DST system using the same evaluation function. A summary table is then created to compare the models using accuracy, F1-score, precision, and recall. The fusion debugging section displays a small sample of SVM attack beliefs, RF attack beliefs, DST attack beliefs, final predictions, and true labels. The last lines compare Random Forest errors with DST errors to identify cases where the DST fusion process corrects Random Forest mistakes.

This section documents the practical execution flow and debugging outputs. Detailed analysis of final numerical results and comparisons will be presented in Chapter Five, which is dedicated to results and discussion.

4-9. Chapter Summary

This chapter presented the practical implementation of the proposed hybrid intrusion detection system. It described the implementation environment, dataset loading, data cleaning, label preparation, class balancing using SMOTE, feature selection using LASSO, feature scaling, SVM training, Random Forest training, probability extraction, Dempster-Shafer fusion, and evaluation. The code screenshots were divided according to their functional role, and each part was explained in relation to the methodology described in Chapter Three. The next chapter should present the experimental results, compare the individual and hybrid models, and discuss the effectiveness of the proposed approach.

Chapter Five: Results and Discussion

5-1. Introduction

This chapter presents the experimental results obtained from the practical implementation of the hybrid intrusion detection system. The evaluation focuses on comparing the individual SVM model, the individual Random Forest model, and the hybrid SVM + RF + DST model. The comparison is based on standard IDS evaluation metrics, including accuracy, precision, recall, F1-score, and confusion matrix analysis.

The purpose of this chapter is to determine whether the evidence-based fusion mechanism improves the final detection decision compared with the individual classifiers. The results are also compared with the primary study adopted in this report in order to evaluate the consistency and practical value of the implemented approach [1].

5-2. Dataset and Processing Output

5-2-1. Dataset Split and Class Distribution

The implementation used the CSE-CIC-IDS2018 dataset after converting the original labels into a binary classification problem. Benign traffic was represented as class 0, while all attack categories were represented as class 1. The output showed that the dataset included several attack categories, such as Infiltration, DDoS attacks, DoS attacks, Bot, SSH-BruteForce, FTP-BruteForce, Brute Force-Web, Brute Force-XSS, and SQL Injection.

After splitting the data, the training subset contained 1,247,454 records with 78 features, while the testing subset contained 311,864 records with 78 features. The class distribution in the training subset was balanced, with 623,727 benign samples and 623,727 attack samples. Therefore, SMOTE did not change the class distribution in this execution, but it remained part of the pipeline to preserve consistency with the adopted methodology.

5-2-2. Feature Selection Output

LASSO feature selection was applied after data preparation. The output showed that the full training matrix originally contained 78 features, then LASSO reduced the selected feature space to 46 features. This confirms that the implementation followed the same methodological direction described in Chapter Three and in the primary paper, where LASSO was used to retain the most discriminative traffic features [1].

The displayed top features included Dst Port, Protocol, Flow Duration, Tot Fwd Pkts, Tot Bwd Pkts, TotLen Fwd Pkts, TotLen Bwd Pkts, Fwd Pkt Len Max, Fwd Pkt Len Min, and Fwd Pkt Len Mean. These features are meaningful for IDS because they describe packet statistics, protocol behavior, and flow-level characteristics that can distinguish normal and malicious traffic.

5-3. Performance Results of the Implemented Models

Table 6 presents the evaluation results of the implemented models. The results show that the hybrid SVM + RF + DST model achieved the best overall performance in terms of accuracy, precision, and F1-score. Random Forest achieved a slightly higher recall than the hybrid model, but the difference was very small.

The hybrid model was therefore the most balanced model in the implemented experiment.

Table 6: Performance Results of the Implemented Models

Model	Accuracy	Precision	Recall	F1-score
SVM Individual	0.903256	0.867046	0.952582	0.907804
Random Forest Individual	0.967717	0.980290	0.954628	0.967289
SVM + RF + DST Hybrid	0.970692	0.986421	0.954525	0.970211

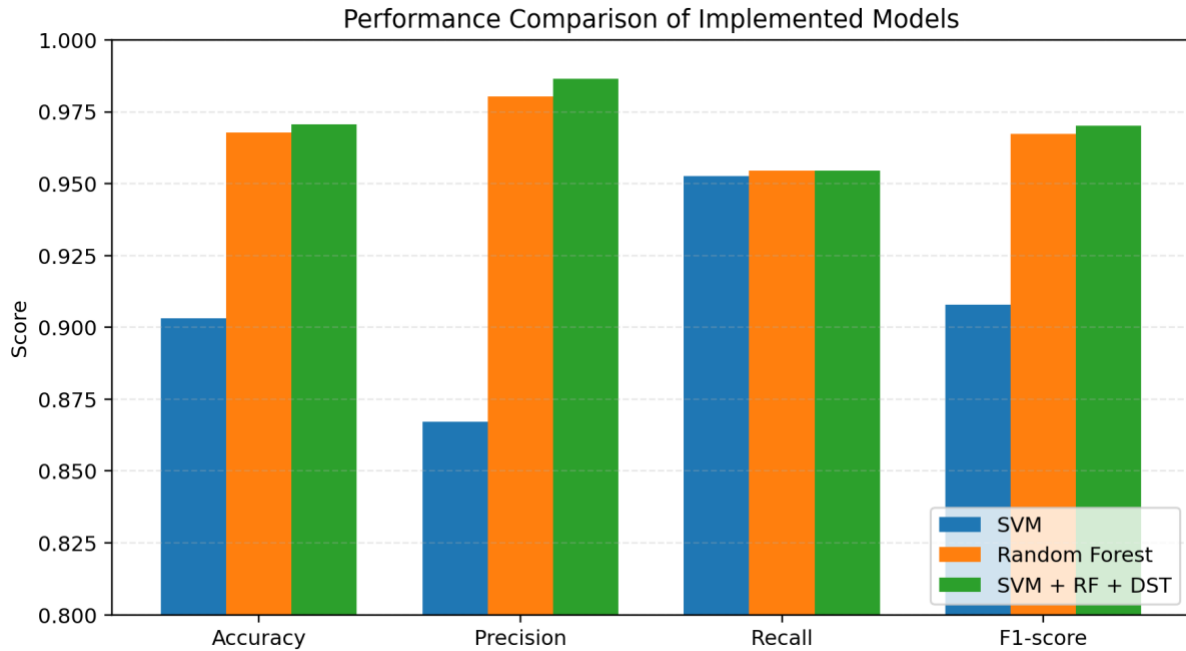


Figure 22: Performance Comparison of Implemented Models

5-3-1. SVM Individual Results

The SVM model achieved 90.32% accuracy, 86.70% precision, 95.26% recall, and 90.78% F1-score. The high recall indicates that SVM detected a large portion of attack samples. However, the lower precision shows that SVM produced a relatively high number of false positives. In an IDS environment, this means that many benign traffic records were incorrectly classified as attacks, which may increase the number of unnecessary alerts. These results can be explained by the use of LinearSVC in the practical implementation. LinearSVC is computationally more suitable for large datasets, but it is less flexible than kernel-based SVM models in capturing complex nonlinear relationships in network traffic.

5-3-2. Random Forest Individual Results

The Random Forest model achieved stronger results than SVM, with 96.77% accuracy, 98.03% precision, 95.46% recall, and 96.73% F1-score. The main improvement appeared in precision, which indicates that Random Forest reduced false alarms significantly compared with SVM. This confirms the suitability of RF for handling nonlinear feature relationships and complex network traffic patterns.

5-3-3. Hybrid SVM-RF-DST Results

The hybrid SVM + RF + DST model achieved the best overall performance, with 97.07% accuracy, 98.64% precision, 95.45% recall, and 97.02% F1-score. Although its recall was slightly lower than the individual RF model, the hybrid model improved both precision and F1-

score. This indicates that DST fusion helped reduce false positives while maintaining a high attack-detection capability.

5-4. Confusion Matrix Analysis

Table 7 summarizes the confusion matrix values for the three implemented models. The values are interpreted as True Negative, False Positive, False Negative, and True Positive. Since the dataset is binary, class 0 represents benign traffic and class 1 represents attack traffic.

Table 7: Confusion Matrix Values of the Implemented Models

Model	TN	FP	FN	TP
SVM Individual	133155	22777	7394	148538
Random Forest Individual	152939	2993	7075	148857
SVM + RF + DST Hybrid	153883	2049	7091	148841

Confusion Matrix: SVM

	Predicted Benign (0)	Predicted Attack (1)
Actual Benign (0)	133155	22777
Actual Attack (1)	7394	148538

Figure 23: Confusion Matrix of the SVM Model

Figure 23 shows that the SVM model detected most attack samples correctly, but it produced 22,777 false positives. This high number of false positives explains the lower precision of the SVM model compared with the other models.

Confusion Matrix: Random Forest

	Predicted Benign (0)	Predicted Attack (1)
Actual Benign (0)	152939	2993
Actual Attack (1)	7075	148857

Figure 24: Confusion Matrix of the Random Forest Model

Figure 24 shows that Random Forest reduced false positives from 22,777 in SVM to 2,993. This reduction explains the significant improvement in precision and overall accuracy. The number of false negatives remained close to the SVM result, which means that RF improved reliability without severely weakening attack detection.

Confusion Matrix: SVM + RF + DST

	Predicted Benign (0)	Predicted Attack (1)
Actual Benign (0)	153883	2049
Actual Attack (1)	7091	148841

Figure 25: Confusion Matrix of the Hybrid SVM-RF-DST Model

Figure 25 shows that the hybrid model reduced false positives further to 2,049. This was the lowest false-positive value among the three models. The false-negative value increased only slightly compared with Random Forest, from 7,075 to 7,091. Therefore, the hybrid model achieved a better operational balance by reducing false alarms while preserving nearly the same attack recall.

5-5. Error Analysis and Operational Interpretation

The error analysis output showed that Random Forest produced 10,068 incorrect predictions, while the DST-based hybrid model produced 9,140 incorrect predictions. This means that the hybrid system reduced the total number of errors by 928 samples. The output also showed that DST fixed 2,247 Random Forest errors, which confirms that the fusion process corrected a considerable number of cases where the individual RF model was wrong.

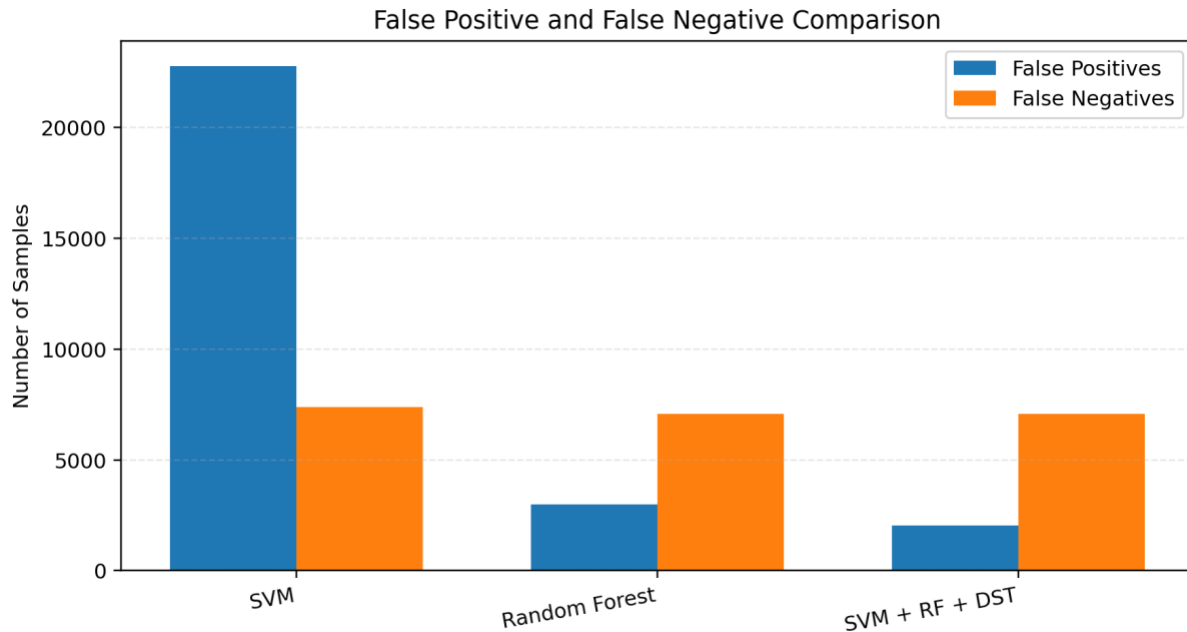


Figure 26: False Positive and False Negative Comparison

Figure 26 highlights the practical improvement achieved by the hybrid model. The main gain appears in the reduction of false positives. In cybersecurity operations, false positives are costly because they generate unnecessary alerts and consume analyst time. Therefore, the reduction from 2,993 false positives in RF to

2,049 in the hybrid model is operationally important. The slight increase in false negatives is small compared with the achieved improvement in precision and total error reduction.

5-6. Comparison with the Primary Study

The implemented results were compared with the primary study, which evaluated SVM, RF, and DST Fusion on the CSE-CIC-IDS2018 dataset. In the primary study, the DST Fusion model achieved the best overall performance, with 0.978380 accuracy, 0.974068 precision, 0.949560 recall, and 0.961658 F1-score [1].

Table 8: Comparison Between Implemented Results and the Primary Study

Model	Metric	Implemented Result	Primary Study Result	Difference
SVM	Accuracy	0.903256	0.977447	-0.074191
SVM	Precision	0.867046	0.970981	-0.103935
SVM	Recall	0.952582	0.949562	+0.003020
SVM	F1-score	0.907804	0.960069	-0.052265
Random Forest	Accuracy	0.967717	0.977257	-0.009540
Random Forest	Precision	0.980290	0.967511	+0.012779
Random Forest	Recall	0.954628	0.952328	+0.002300
Random Forest	F1-score	0.967289	0.959859	+0.007430
SVM + RF + DST	Accuracy	0.970692	0.978380	-0.007688
SVM + RF + DST	Precision	0.986421	0.974068	+0.012353
SVM + RF + DST	Recall	0.954525	0.949560	+0.004965
SVM + RF + DST	F1-score	0.970211	0.961658	+0.008553

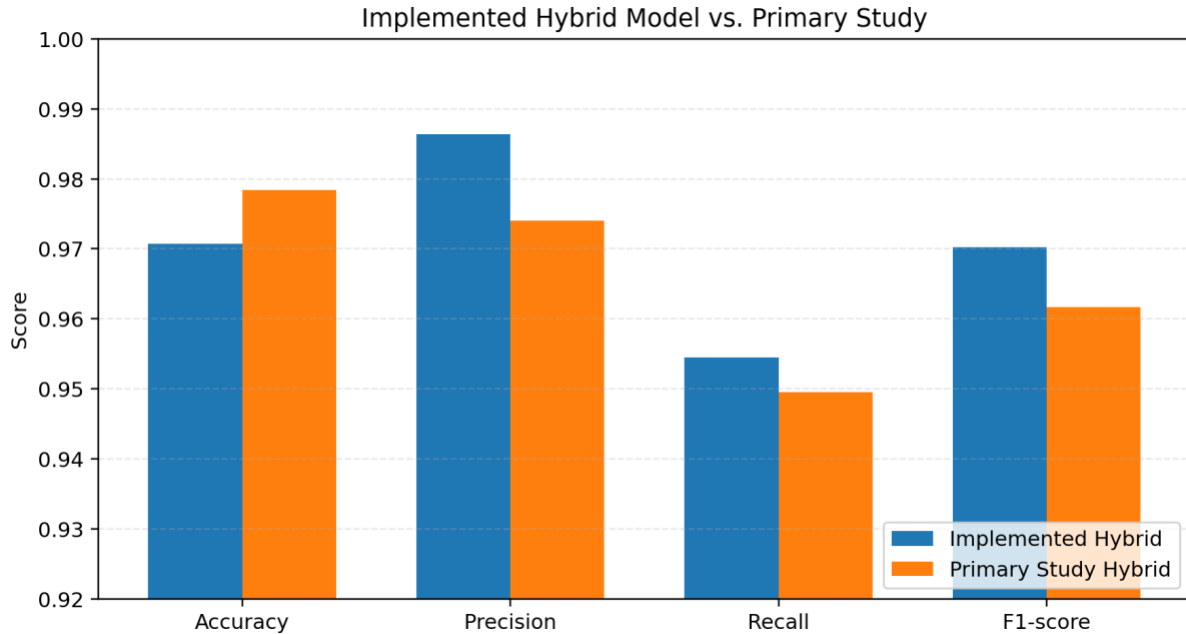


Figure 27: Implemented Hybrid Model Compared with the Primary Study

The comparison shows that the implemented hybrid model followed the same general trend as the primary study: DST fusion produced the strongest overall model. The implemented hybrid model achieved lower accuracy than the primary study, but it achieved higher precision, recall, and F1-score. The difference in accuracy can be explained by differences in implementation scale, local configuration, selected dataset subset, and SVM implementation details. In this project, the test set contained 311,864 records, while the primary study used a substantially larger evaluation scale [1].

The SVM result in this implementation was lower than the SVM result in the primary study. This is expected because the local implementation used a simpler linear SVM configuration with probability calibration, while the primary study described a more optimized SVM configuration. In contrast, RF and the hybrid model achieved competitive results and even exceeded the primary study in precision and F1-score. This confirms that the local implementation successfully reproduced the main benefit of the proposed methodology: improving IDS reliability through evidence-based fusion.

5-7. Discussion

The results demonstrate that SVM alone was less suitable in the current implementation because LinearSVC was used instead of a more general kernel-based SVM. Although LinearSVC is faster and more suitable for the available local environment, it may be less capable of modeling nonlinear attack patterns in network traffic. This explains why SVM produced a larger number of false positives and lower precision compared with Random Forest and the hybrid SVM + RF + DST model. Random Forest provided a strong baseline by significantly improving precision and reducing false alarms. However, the hybrid model provided the best overall result because it further reduced false positives and produced the highest F1-score.

The improvement achieved by DST can be explained by its ability to combine evidence from two classifiers rather than depending on one model. When the classifiers agree, DST strengthens the final decision. When there is disagreement, DST uses the conflict between evidence sources to produce a more reliable final belief-based classification. This makes the hybrid system more appropriate for IDS environments, where traffic behavior may be ambiguous and classifier outputs may not always agree.

The results also show that the implemented pipeline was consistent with the methodology described in Chapter Three. The dataset was prepared as a binary classification task, the training data was checked for class balance, LASSO reduced the feature space from 78 to 46 features, SVM and RF were trained as base classifiers, and DST was used to combine their probabilistic outputs. Therefore, the implementation supports the theoretical claim that classifier fusion can improve IDS robustness.

5-8. Chapter Summary

This chapter presented the experimental results of the implemented hybrid IDS. The SVM model achieved high recall but produced many false positives, which reduced its precision and accuracy. Random Forest produced stronger results and became the best individual classifier. The hybrid SVM + RF + DST model achieved the highest accuracy, precision, and F1-score, and it reduced the total number of errors compared with Random Forest. The comparison with the primary study showed that the implemented system follows the same trend reported in the literature: DST-based fusion improves the reliability of intrusion detection decisions compared with individual classifiers. These results provide a strong basis for the final conclusion of the report.

Conclusion and Future Work

Project Conclusion

This project presented a hybrid Intrusion Detection System based on integrating Support Vector Machine and Random Forest classifiers using Dempster-Shafer Theory. The main objective was to improve the reliability of intrusion detection by combining the complementary strengths of two machine learning classifiers and using an evidence-fusion mechanism to reduce uncertainty in the final decision. The project was implemented and evaluated using the CSE-CIC-IDS2018 dataset, which provides labeled benign and malicious network traffic records.

The proposed system followed a structured methodology that included data preparation, preprocessing, feature selection using LASSO, class balancing verification using SMOTE, training of SVM and Random Forest classifiers, and final decision fusion using Dempster-Shafer Theory. LASSO reduced the feature set from 78 to 46 selected features, which helped reduce dimensionality while preserving the most discriminative network traffic characteristics. The use of feature scaling and data preprocessing also contributed to preparing consistent inputs for the learning models.

Main Contributions

The main contribution of this project is the implementation and analysis of a hybrid IDS framework that combines margin-based classification, ensemble learning, and uncertainty-aware evidence fusion. SVM contributes boundary-based classification, Random Forest contributes ensemble-based robustness, and Dempster-Shafer Theory combines their outputs in a more reliable decision-fusion process.

The experimental results showed that the hybrid SVM-RF-DST model achieved the best overall performance compared with the individual classifiers. The implemented hybrid system achieved 97.07% accuracy, 98.64% precision, 95.45% recall, and 97.02% F1-score. Compared with the individual Random Forest model, the hybrid model reduced the number of false positives from 2,993 to 2,049, which indicates that Dempster-Shafer fusion improved the reliability of intrusion alerts.

Limitations

Although the implemented hybrid model achieved strong results, several limitations should be considered. The local implementation environment was different from the reference experimental environment used in the primary study. In addition, the implemented SVM configuration was simpler than the optimized GPU-accelerated SVM configuration used in the primary study.

Another limitation is that the current implementation is mainly based on batch processing rather than real-time network traffic monitoring. Therefore, the system demonstrates the effectiveness of the proposed pipeline in an experimental setting, but further work is needed

before deploying it as a real-time intrusion detection solution in high-speed or large-scale networks.

Future Work

Future work should focus on extending the system toward practical cybersecurity deployment. The primary study suggests that the current fusion process is mainly optimized for batch-based processing; therefore, one important direction is to develop a real-time or near-real-time IDS implementation capable of processing incoming network flows continuously.

Another important direction is online DST-based fusion, where belief assignments are updated dynamically as new traffic records arrive. Future research may also introduce non-zero uncertainty mass in the DST framework to represent ambiguous network traffic more realistically. In addition, the system can be evaluated on larger-scale and more diverse cybersecurity datasets, tested against unseen or zero-day attack scenarios, and optimized to reduce false alarms while maintaining strong attack-detection capability.

References

- [1] B. Waheed, M. B. Irfansyah, I. Winarno, and A. Alimudin, “Enhancing Intrusion Detection Systems by Integrating Support Vector Machine and Random Forest Classifiers Using Dempster-Shafer Theory,” *Inform: Jurnal Ilmiah Bidang Teknologi Informasi dan Komunikasi* , vol. 11, no. 1, 2026. DOI: 10.25139/inform.v11i1.11075.
- [2] A. Dallali, T. Omrani, and B. C. Rhaimi, “Evidence Theory Data Fusion-Based Method for Cyber-Attack Detection,” *Conference / Research Paper*, 2020.
- [3] P. Galdames, S. Y. Enoch, C. Gutiérrez-Soto, and M. A. Palomino, “A Dempster-Shafer, Fusion-Based Approach for Malware Detection,” *Mathematics* , vol. 13, no. 16, article 2677, 2025. DOI: 10.3390/math13162677.
- [4] V. Sharma and D. J. Shah, “Ensemble Learning Classifiers and Hybrid Feature Selection for Enhancing Intrusion Detection System Performance,” *Journal of Information Systems Engineering and Management* , vol. 10, no. 15s, 2025.
- [5] P. Dey and D. Bhakta, “A New Random Forest and Support Vector Machine-Based Intrusion Detection Model in Networks,” *National Academy Science Letters* , 2023.
- [6] C. Liu, Z. Gu, and J. Wang, “A Hybrid Intrusion Detection System Based on Scalable K-Means, Random Forest and Deep Learning,” *IEEE Access* , 2021.
- [7] H. Shakeri, H. Raei, and E. Nejati, “Improving Accuracy in Intrusion Detection Systems Using Classifier Ensemble and Clustering,” *Journal of Advances in Computer Research* , 2020.
- [8] C. Bhatt, “Improving Intrusion Detection Systems with Artificial Intelligence: A Review of Techniques and Applications,” *Turkish Journal of Computer and Mathematics Education* , 2019.
- [9] S. M. Yellepeddi, “AI-Powered Intrusion Detection Systems: Real-World Performance Analysis,” *Journal of AI-Assisted Scientific Discovery* , 2024.

